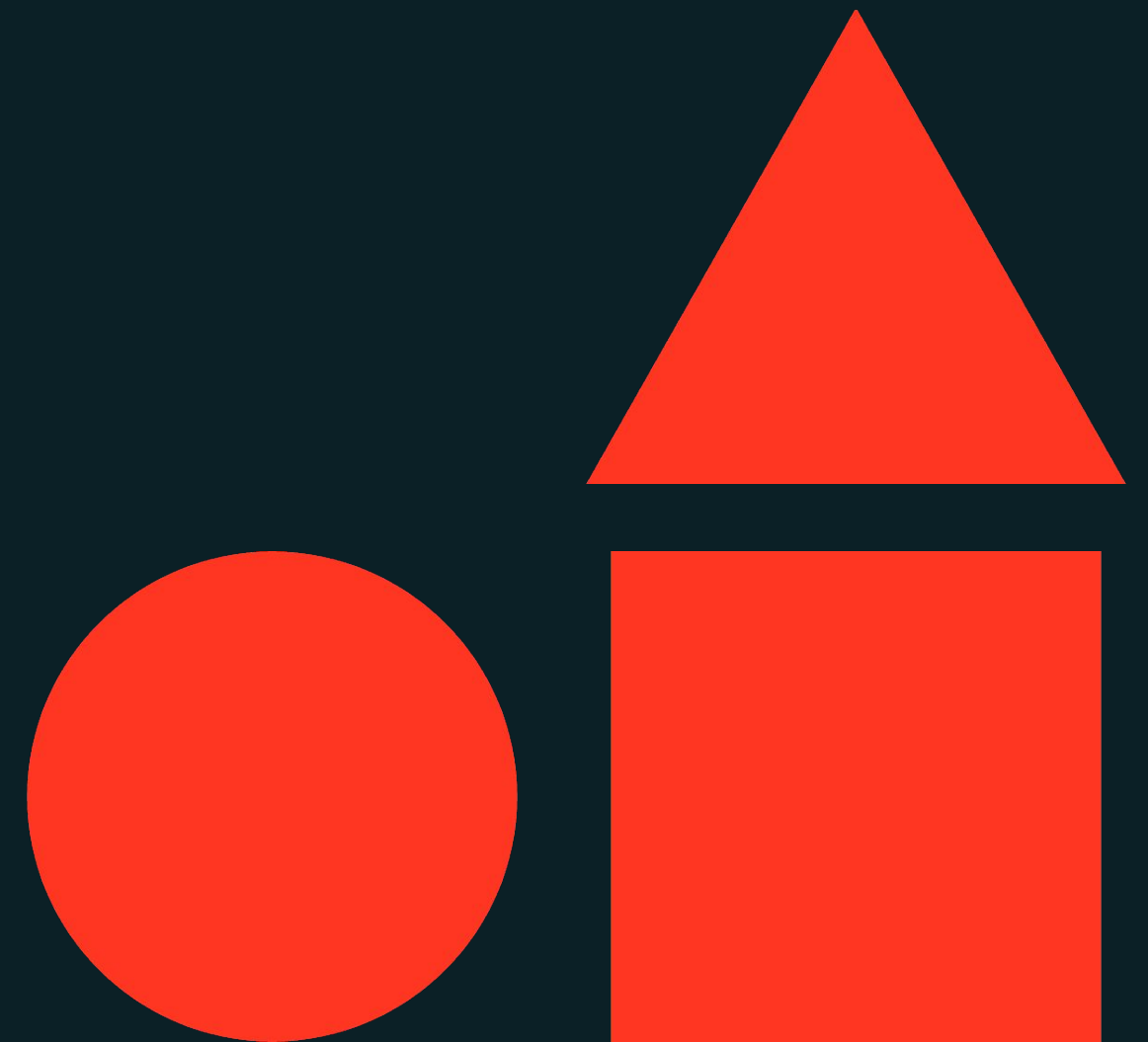




Deploy Workloads with Lakeflow Jobs

Databricks Academy



Course Learning Objectives

- Understand the role of Lakeflow Jobs within the Databricks ecosystem as a unified orchestration platform for data, analytics, and AI workloads.
- Design and implement data workloads using Directed Acyclic Graphs (DAGs), demonstrating the relationship between jobs, tasks, and dependencies.
- Configure various job scheduling options including manual, scheduled, file arrival, and continuous triggers to automate workflow execution.
- Implement advanced workflow features such as conditional task execution, run-if dependencies, and repair runs to create robust, fault-tolerant data pipelines.
- Apply best practices for production workloads including selecting appropriate compute options, implementing modular orchestration, and utilizing proper error handling techniques.



Agenda

Course Sections

- **Introduction to Data Engineering in Databricks**
- **Lakeflow Jobs Core Concepts**
- **Creating and Scheduling Jobs**
- **Advance Lakeflow Jobs Features**



Course Prerequisites (REQUIRED)



Fundamental Knowledge of the Databricks Platform

- Databricks Workspaces
- Apache Spark
- Delta Lake and the Medallion Architecture
- Unity Catalog



Course Prerequisites (REQUIRED)



Fundamental Knowledge of the Databricks Platform

- Databricks Workspaces
- Apache Spark
- Delta Lake and the Medallion Architecture
- Unity Catalog



Basic Knowledge of Python

- Familiarity with basic knowledge of python, pyspark or spark.sql



Course Prerequisites (REQUIRED)



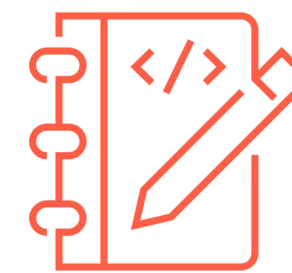
Fundamental Knowledge of the Databricks Platform

- Databricks Workspaces
- Apache Spark
- Delta Lake and the Medallion Architecture
- Unity Catalog



Basic Knowledge of Python

- Familiarity with basic knowledge of python, pyspark or spark.sql



Experience Transforming Data with SQL

- Experience in writing **intermediate-level queries** using SQL



Lab Exercise Environment

Technical Details



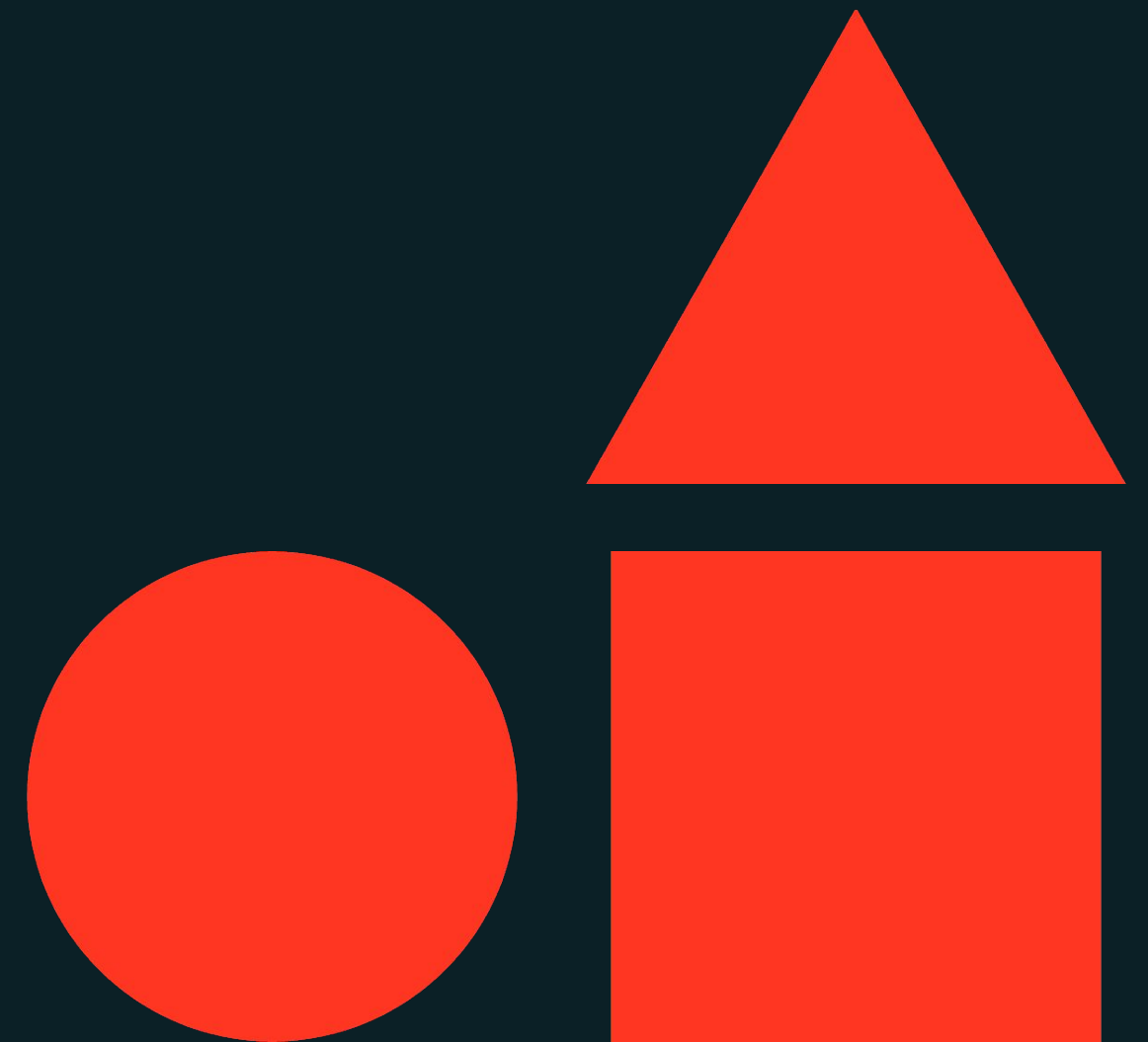
- Your lab environment is provided by Vocareum.
- It will open in a new tab.
- It has been configured with the permissions and resources required to accomplish the tasks outlined in the lab exercise.
- Third party cookies must be enabled in your browser for Vocareum's user experience to work properly.
- Make sure to enable pop ups!





Introduction to Data Engineering in Databricks

Deploy Workloads with Lakeflow Jobs



Section Learning Objectives

- Understand the key components of data engineering on the Databricks platform, including Connect, Delta Live Tables (DLT), and Jobs.
- Explain what Lakeflow Jobs are and their main benefits for unified orchestration of data, analytics, and AI workloads.
- Identify the core capabilities and use cases enabled by Lakeflow Jobs within the Databricks ecosystem.



Agenda

Introduction to Data Engineering in Databricks

- **Data Engineering in Databricks**
- **What is Lakeflow Jobs?**





Introduction to Data Engineering in
Databricks

LECTURE

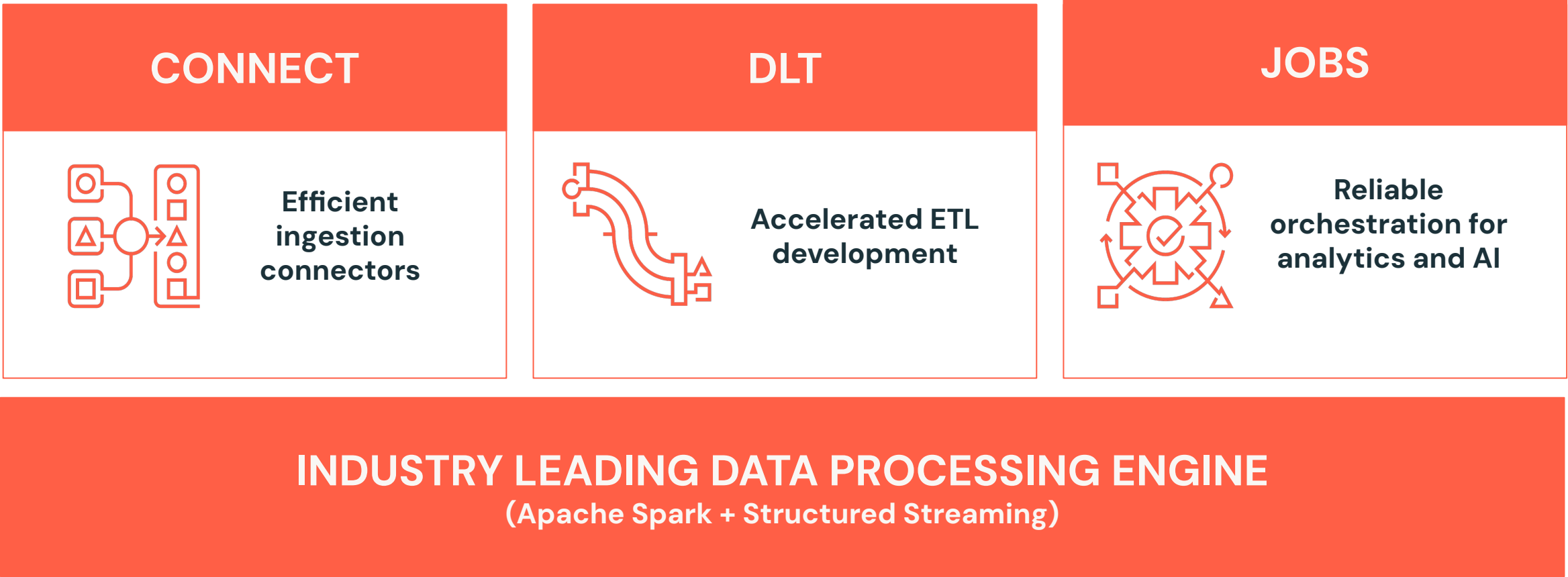
Data Engineering in Databricks



Data Engineering in Databricks



UNIFIED DATA ENGINEERING FOR THE DATA INTELLIGENCE PLATFORM



UNIFIED GOVERNANCE



OPTIMIZED STORAGE



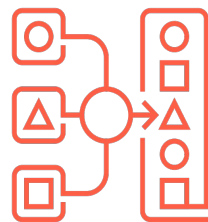
Data Engineering in Databricks



databricks LAKEFLOW

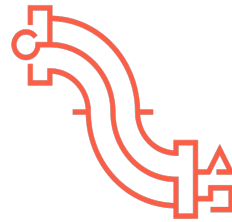
UNIFIED DATA ENGINEERING FOR THE DATA INTELLIGENCE PLATFORM

CONNECT



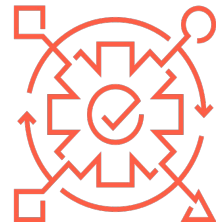
Efficient
ingestion
connectors

DLT



Accelerated ETL
development

JOB



Reliable
orchestration for
analytics and AI

INDUSTRY LEADING DATA PROCESSING ENGINE
(Apache Spark + Structured Streaming)

UNIFIED GOVERNANCE



Unity Catalog

OPTIMIZED STORAGE



DELTA LAKE



Parquet ICEBERG

Orchestrate every workload
(Notebooks, SQL, dashboards,
Pipelines, and more)





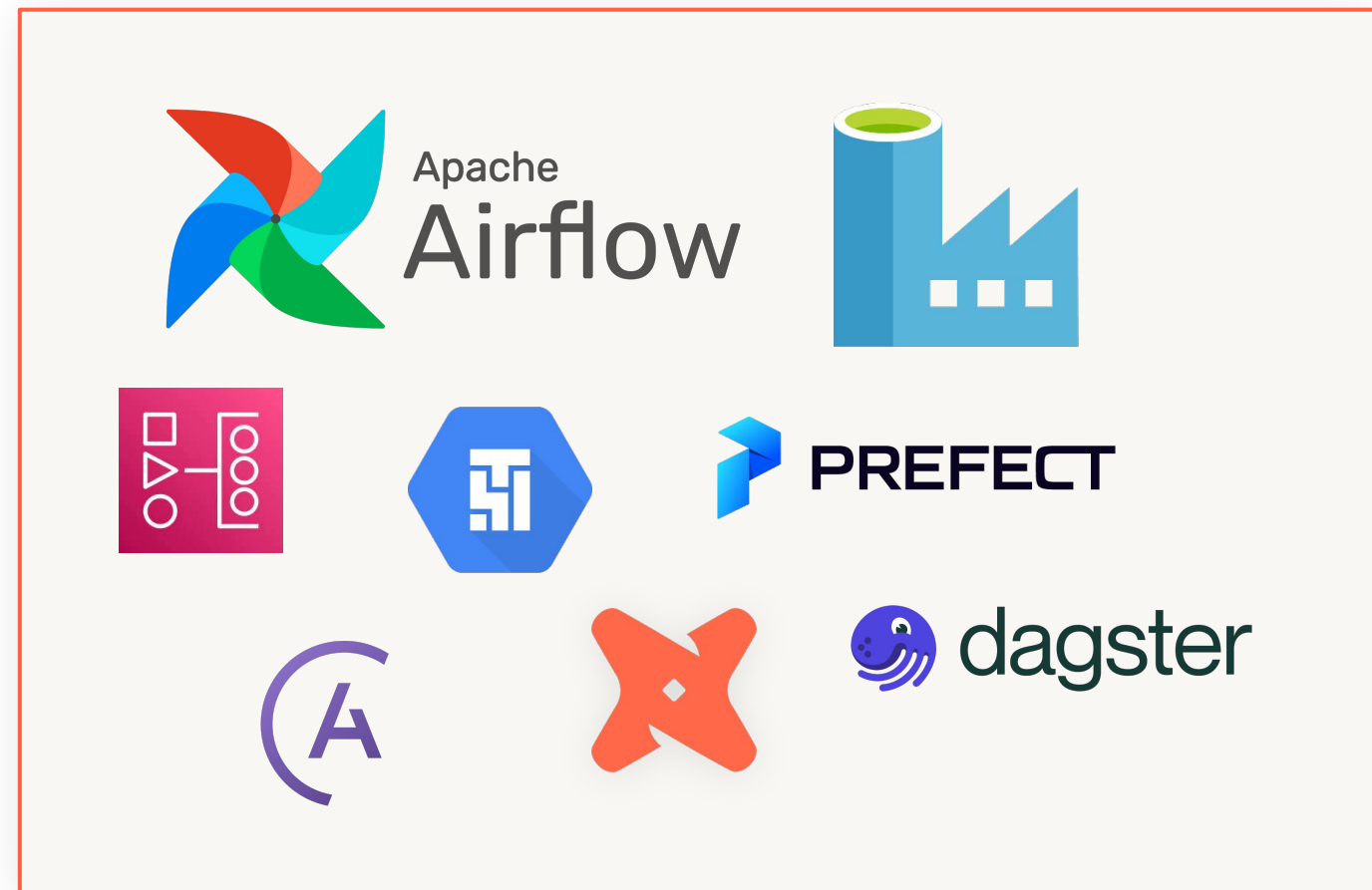
Introduction to Data Engineering in
Databricks

LECTURE

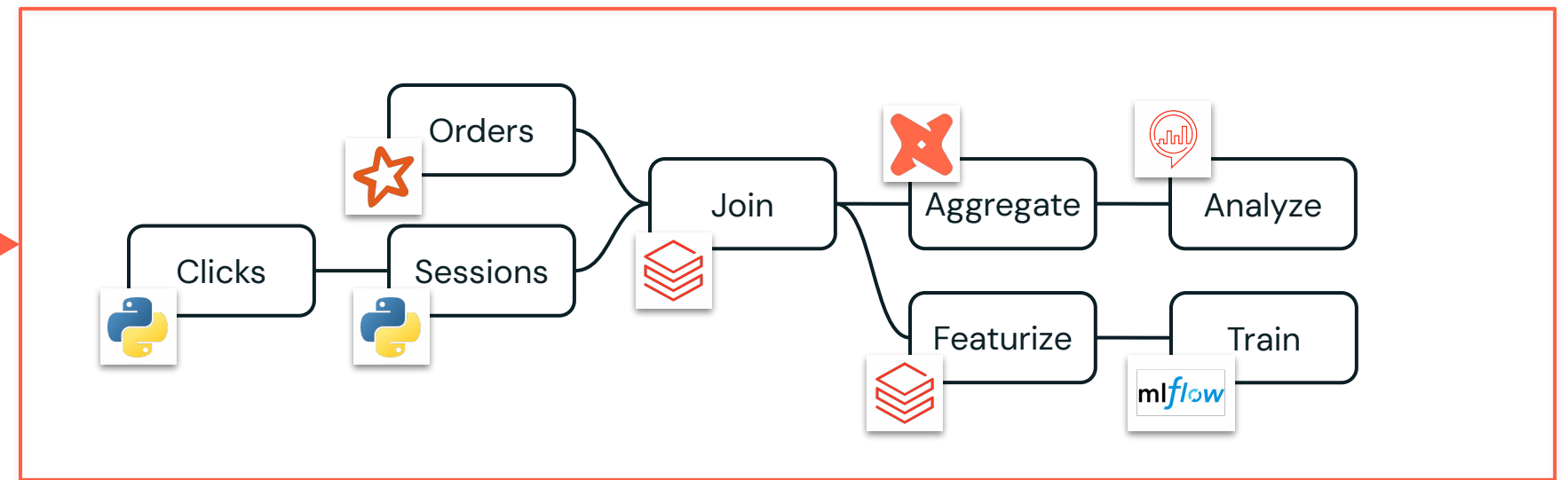
What is Lakeflow Jobs?



There are many ways to orchestrate your workloads



 databricks Data Intelligence Platform




BI & Data
Warehousing


Data
Engineering


Data
Streaming


Data
Science & ML

External orchestrators create challenges

Hard to use for many practitioners



Data teams are less productive

Difficult to understand root cause when issues occur

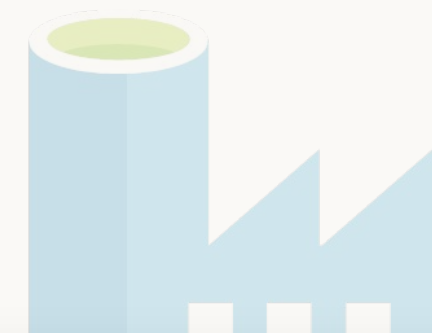


Bad data lowers value of downstream applications

Complex architecture to manage and maintain



Higher cost of ownership and lower reliability

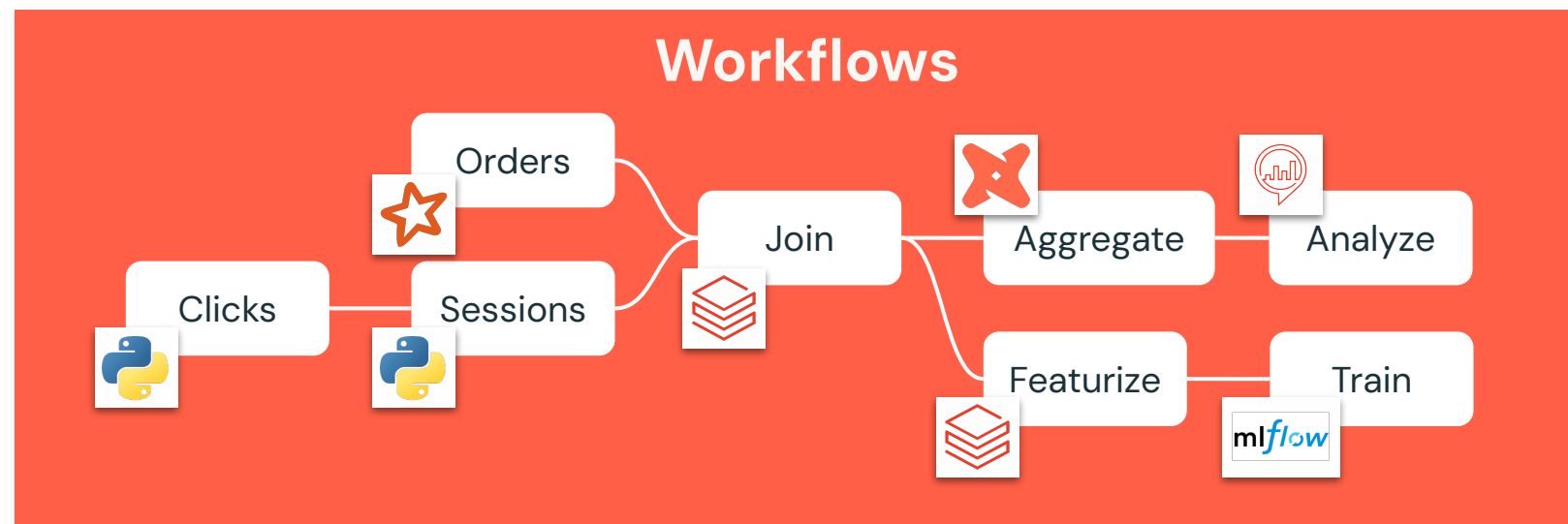


These tools are not unified with your Lakehouse



What is Lakeflow Jobs?

 **databricks** Data Intelligence Platform



Data Ingestion
& Transformation

Data
Warehousing

Machine
Learning

Processing Engine (Photon)

Governance (Unity Catalog)

Storage (Delta Lake)

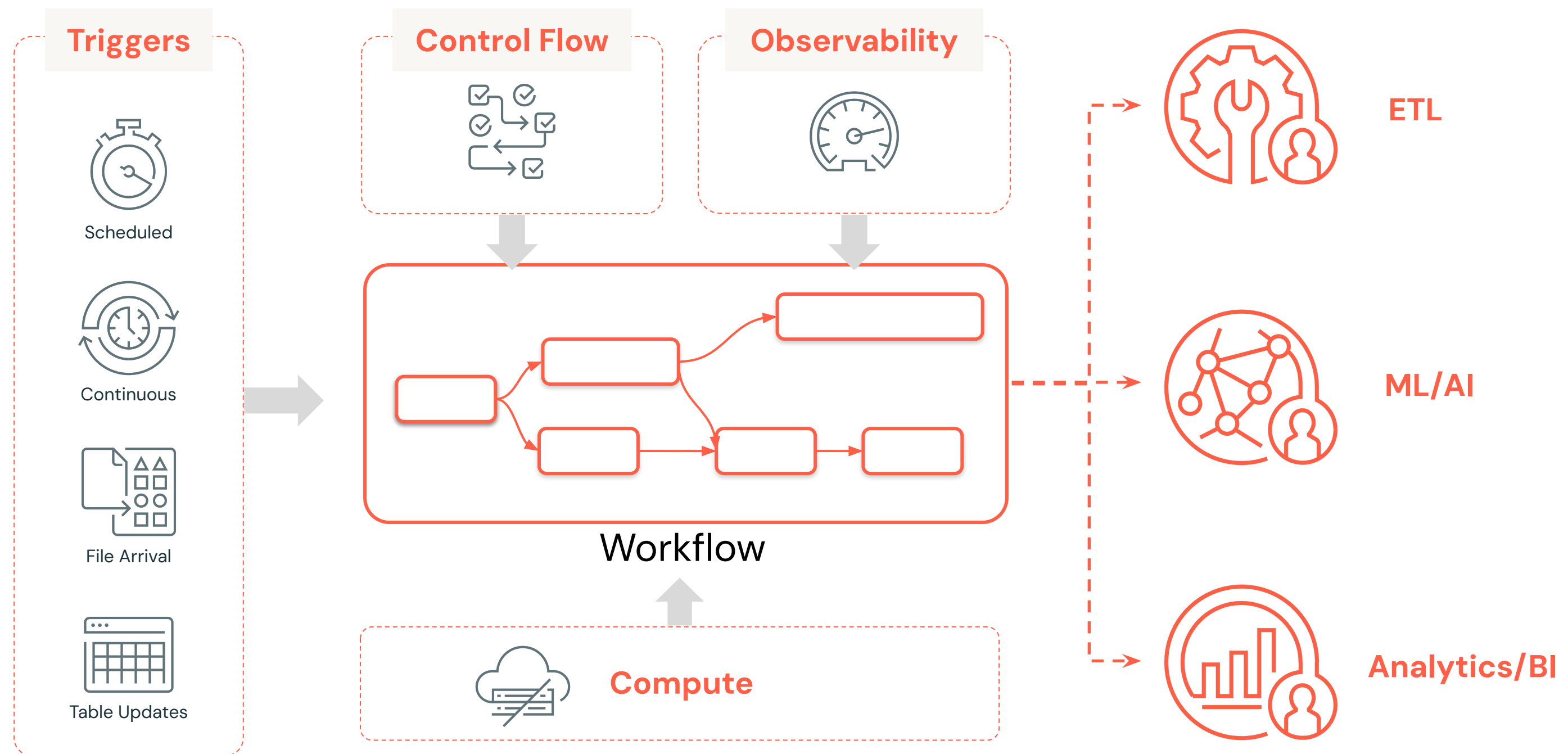


Unified orchestration for data, analytics,
and AI on the Data Intelligence Platform

- Simple authoring
- Actionable insights
- Proven reliability



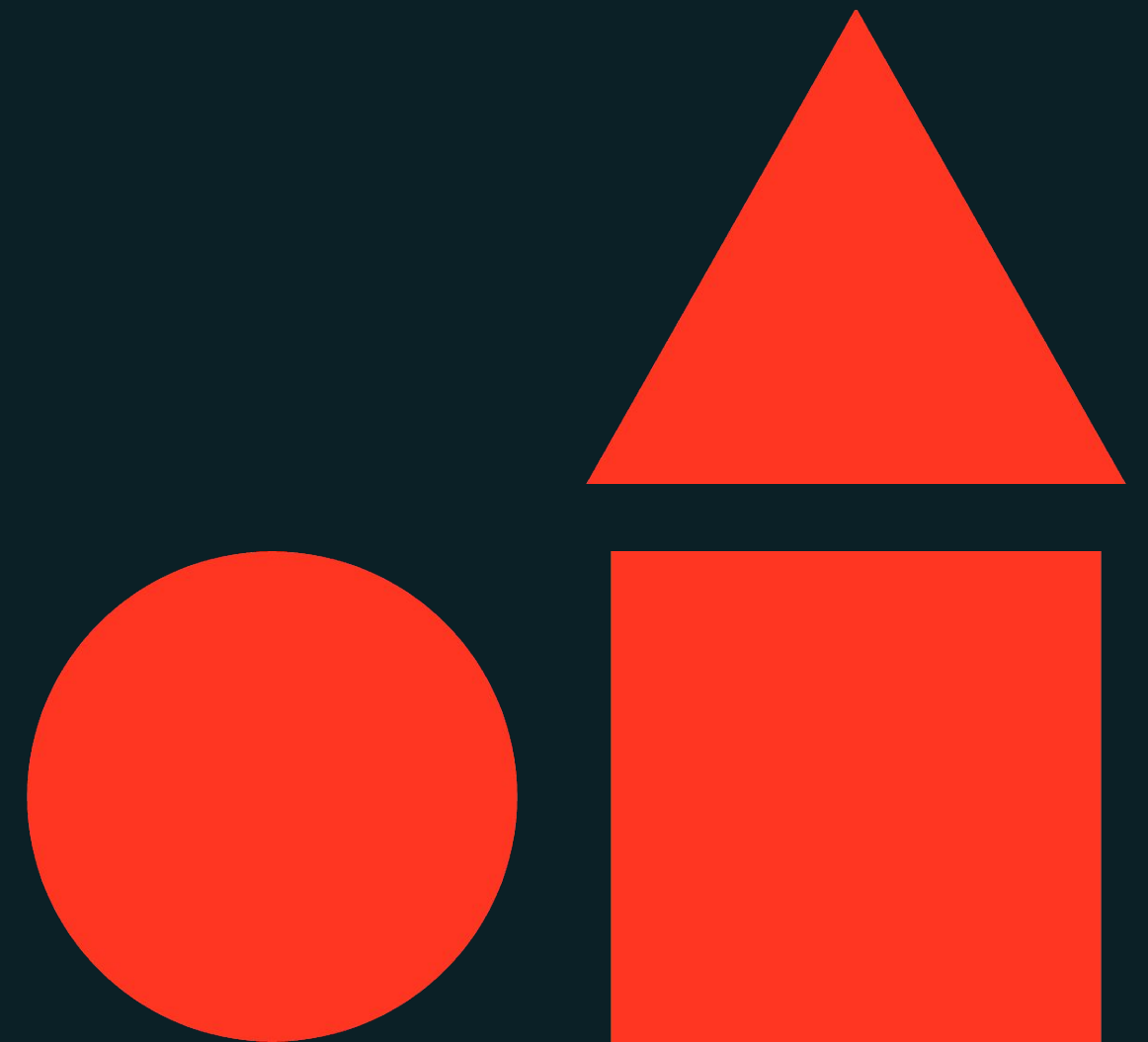
Lakeflow Jobs





Lakeflow Jobs Core Components

Deploy Workloads with Lakeflow Jobs



Section Learning Objectives

- Understand the building blocks of Lakeflow Jobs, including the concepts of jobs and tasks.
- Identify and describe the various task types and configuration options available in Lakeflow Jobs.
- Explain how Directed Acyclic Graphs (DAGs) enable task orchestration and workflow patterns.
- Learn how to create and configure jobs using the Lakeflow Jobs UI.



Agenda

Section Overview – Lakeflow Jobs Core Components

- **Building Blocks of Lakeflow Jobs**
- **DAG and Task Orchestration**
- **Creating a Job Using Lakeflow Jobs UI**





Lakeflow Jobs Core Components

LECTURE

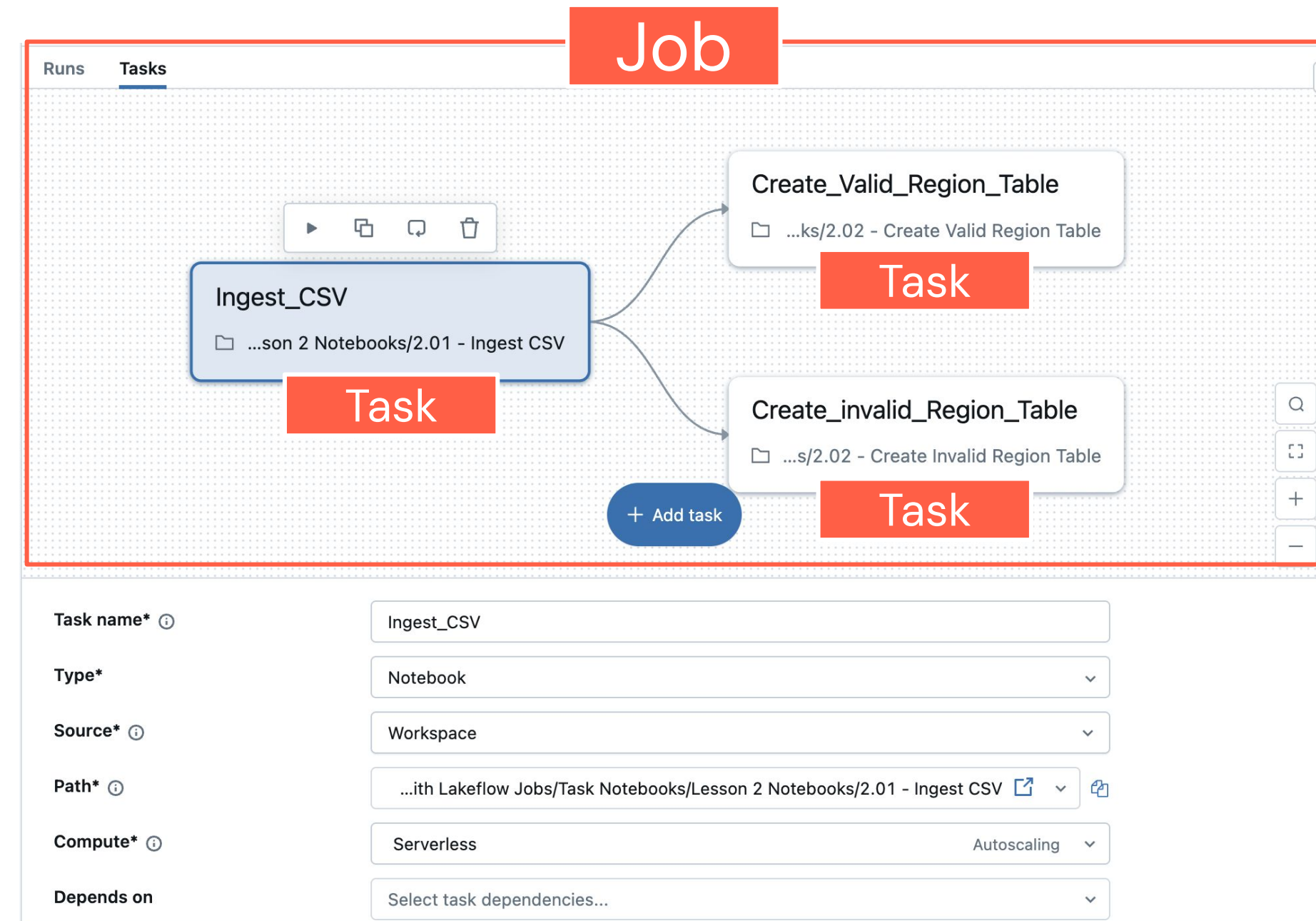
Building Blocks of Lakeflow Jobs



Building Blocks of Lakeflow Jobs

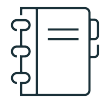
Job and Tasks

- A **Job** is the primary resource for:
 - Scheduling
 - Coordinating
 - Running operations such as data processing, ETL, analytics, and machine learning workloads within the Databricks environment.
- A **task** is a **single unit of work** within a job that executes a specific workload such as a notebook, script, query and more.
- Each **job** consists of **one or more tasks**, which are the individual units of work that make up the job



Building Blocks of Lakeflow Jobs

Jobs consist of one or more Tasks



Databricks Notebooks



DLT



SQL Files/
Queries



Python
Scripts



Python
Wheels



AI/BI
Dashboards



Spark
Submit



dbt



Java
JAR file



Power BI

Code

Notebook

Python script

Python wheel

JAR

Spark Submit

Clean Room notebook

SQL

Legacy dashboard

SQL query

SQL alert

SQL file

Ingestion and transformation

Pipeline

Lakeflow Jobs ☆

Runs Tasks

▶ □ 🗑

Unnamed task

📁 Unspecified path

🔍

📐

+

-

Task name* ⓘ

Type* Notebook ▾

Source* ⓘ Workspace ▾

Path* ⓘ Select Notebook ▾

Compute* ⓘ Serverless Autoscaling ▾

Dependent libraries ⓘ + Add



Building Blocks of Lakeflow Jobs

Job Tasks

- With **Jobs**, we can set a **particular task**
- We have different **options available** based on the task:
 - defining path
 - adding libraries
 - parameters
 - enabling notification
 - etc to better orchestrate particular task

Lakeflow Jobs ☆

Runs **Tasks**

Set Specific Task

▶ □ 🗑️

Unnamed task
📁 Unspecified path

Task Options

Task name* ⓘ

Type*

Source* ⓘ

Path* ⓘ

Compute* ⓘ Autoscaling

Dependent libraries ⓘ

Parameters ⓘ

Key Value

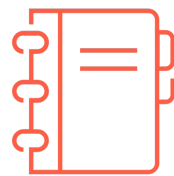
Cancel Create task

Building Blocks of Lakeflow Jobs

Job Tasks – Notebook

Based on **task type selected**, you get specific options

Example



For **Notebook** Tasks:

- Source
- Path
- Compute options
- And More

The screenshot shows the 'Lakeflow Jobs' interface with the 'Tasks' tab selected. A task named 'Unnamed task' is shown with an 'Unspecified path'. Below the task list, a configuration form is visible, outlined in red. The form includes the following fields:

- Task name***: A text input field.
- Type***: A dropdown menu set to 'Notebook'.
- Source***: A dropdown menu set to 'Workspace'.
- Path***: A dropdown menu set to 'Select Notebook'.
- Compute***: A dropdown menu set to 'Serverless' with an 'Autoscaling' sub-menu.
- Dependent libraries**: A section with a '+ Add' button.
- Parameters**: A section with 'UI' and 'JSON' tabs.

At the bottom right of the form are 'Cancel' and 'Create task' buttons.

Building Blocks of Lakeflow Jobs

Job Tasks – SQL

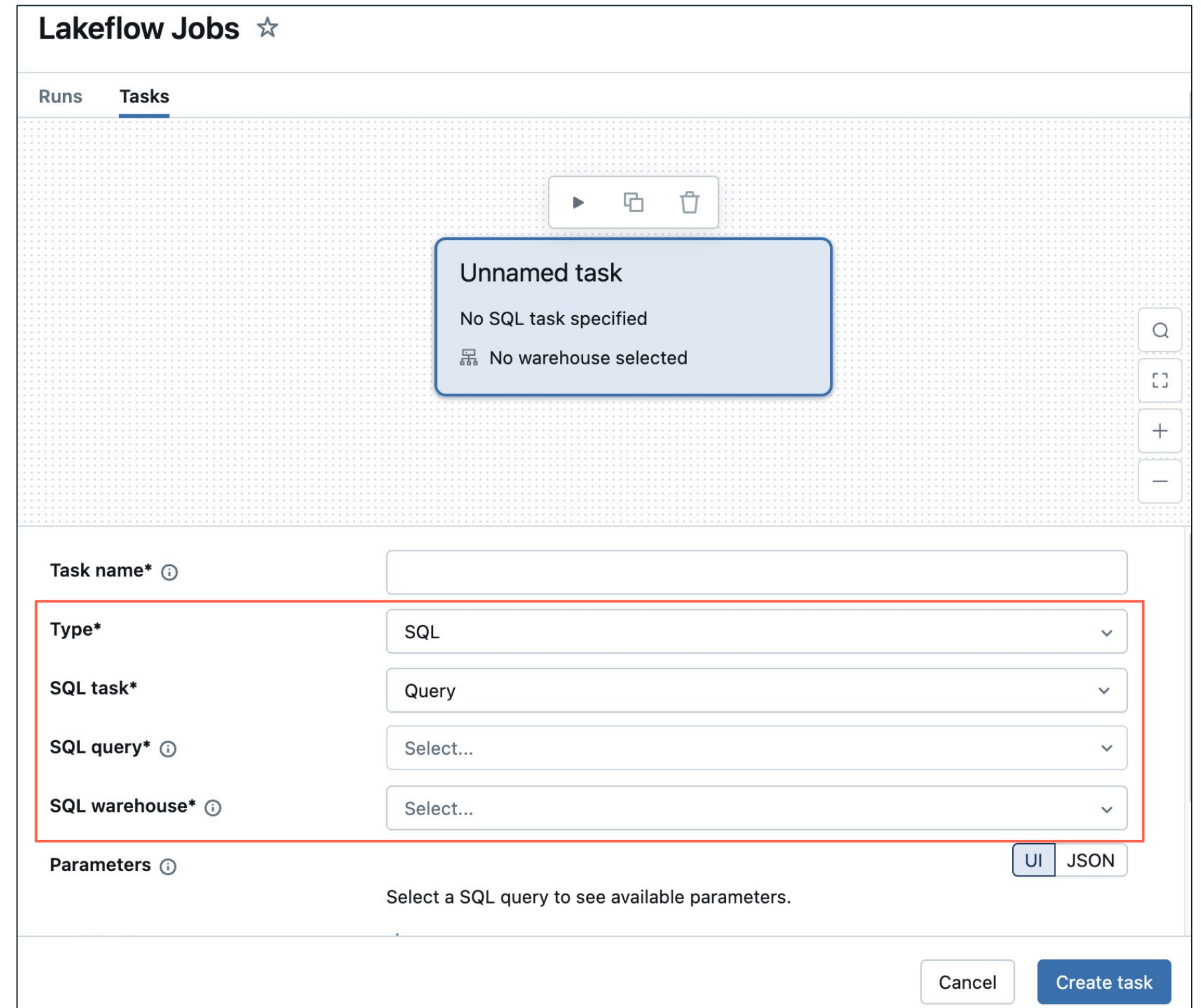
Based on **task type selected**, you get specific options

Example



For **SQL** Tasks:

- SQL task
- SQL query
- SQL warehouse

A screenshot of the Lakeflow Jobs web interface. The 'Tasks' tab is active. A modal box titled 'Unnamed task' displays error messages: 'No SQL task specified' and 'No warehouse selected'. Below the modal, a form for creating a task is shown. The 'Task name*' field is empty. The 'Type*' dropdown is set to 'SQL'. The 'SQL task*' dropdown is set to 'Query'. The 'SQL query*' and 'SQL warehouse*' dropdowns are both set to 'Select...'. A red rectangle highlights the 'Type*', 'SQL task*', 'SQL query*', and 'SQL warehouse*' fields. At the bottom, there are 'Parameters' tabs for 'UI' and 'JSON', a message 'Select a SQL query to see available parameters.', and 'Cancel' and 'Create task' buttons.

Lakeflow Jobs ☆

Runs Tasks

Unnamed task

No SQL task specified

No warehouse selected

Task name* ⓘ

Type* SQL ▼

SQL task* Query ▼

SQL query* ⓘ Select... ▼

SQL warehouse* ⓘ Select... ▼

Parameters ⓘ UI JSON

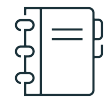
Select a SQL query to see available parameters.

Cancel Create task



Building Blocks of Lakeflow Jobs

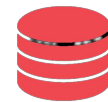
Jobs consist of one or more **Tasks**



Databricks Notebooks



DLT



SQL Files/
Queries



Python
Scripts



Python
Wheels



AI/BI
Dashboards



Spark
Submit



dbt



Java
JAR file

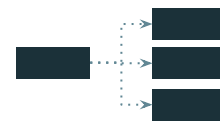


Power BI

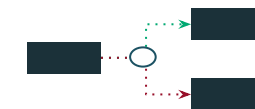
Control flows can be established between **Tasks**



Sequential



Parallel



Conditionals
(If/else)



Run Job
(Modular)



For each

Jobs supports different **Triggers**



Scheduled
(Cron)



API
Trigger



File
Arrival Triggers



Table
Triggers



Continuous
(Streaming)



Manual
Trigger

COMPUTE

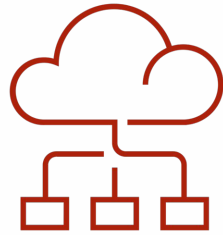


Building Blocks of Lakeflow Jobs

Jobs can be executed on different compute

Development, Ad-Hoc Analysis & Exploration

Interactive Clusters



Interactive or all-purpose clusters can be shared by multiple users.

It is best for performing **ad-hoc analysis, data exploration, or development.**

Interactive **should not be used in Production** as they are not cost-efficient.



Building Blocks of Lakeflow Jobs

Jobs can be executed on different compute

Development, Ad-Hoc Analysis & Exploration

Interactive Clusters



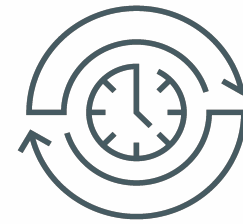
Interactive or all-purpose clusters can be shared by multiple users.

It is best for performing **ad-hoc analysis, data exploration, or development.**

Interactive **should not be used in Production** as they are not cost-efficient.

Production Grade & Operational Use-Cases

Job Clusters



Job clusters are approximately **50% cheaper** as they terminate when the job ends, reducing resource usage and costs.

However, Job clusters are subject to **cloud providers start-up times.**

With Databricks Jobs, you can **re-use the same cluster across Tasks** for **better price performance!**



Building Blocks of Lakeflow Jobs

Jobs can be executed on different compute

Development, Ad-Hoc Analysis & Exploration

Interactive Clusters



Interactive or all-purpose clusters can be shared by multiple users.

It is best for performing **ad-hoc analysis, data exploration, or development**.

Interactive **should not be used in Production** as they are not cost-efficient.

Production Grade & Operational Use-Cases

Job Clusters



Job clusters are approximately **50% cheaper** as they terminate when the job ends, reducing resource usage and costs.

However, Job clusters are subject to **cloud providers start-up times**.

With Databricks Jobs, you can **re-use the same cluster across Tasks** for **better price performance!**

Simple, Fast, Reliable, Cost Efficient!

Serverless



Serverless Workflows are a **fully managed service** that are operationally simpler and more reliable

They provide you with **faster clusters & auto-scaling capabilities** providing you with a **better user experience for a lower cost**.

With **out of the box performance optimizations**, Serverless provides you with a **lower overall TCO**.



Building Blocks of Lakeflow Jobs

Languages Supported

- Python
- SQL
- Scala
- R
- Java [via a JAR file]





Lakeflow Jobs Core Components

LECTURE

DAG and Task Orchestration



DAG and Task Orchestration

What is a DAG?

A DAG is a **conceptual representation** of series of activities, including data processing flows.

- **Directed** – unambiguous direction for each edge
- **Acyclic** – contains no cycles
- **Graph**– collection of vertices connected by edges.

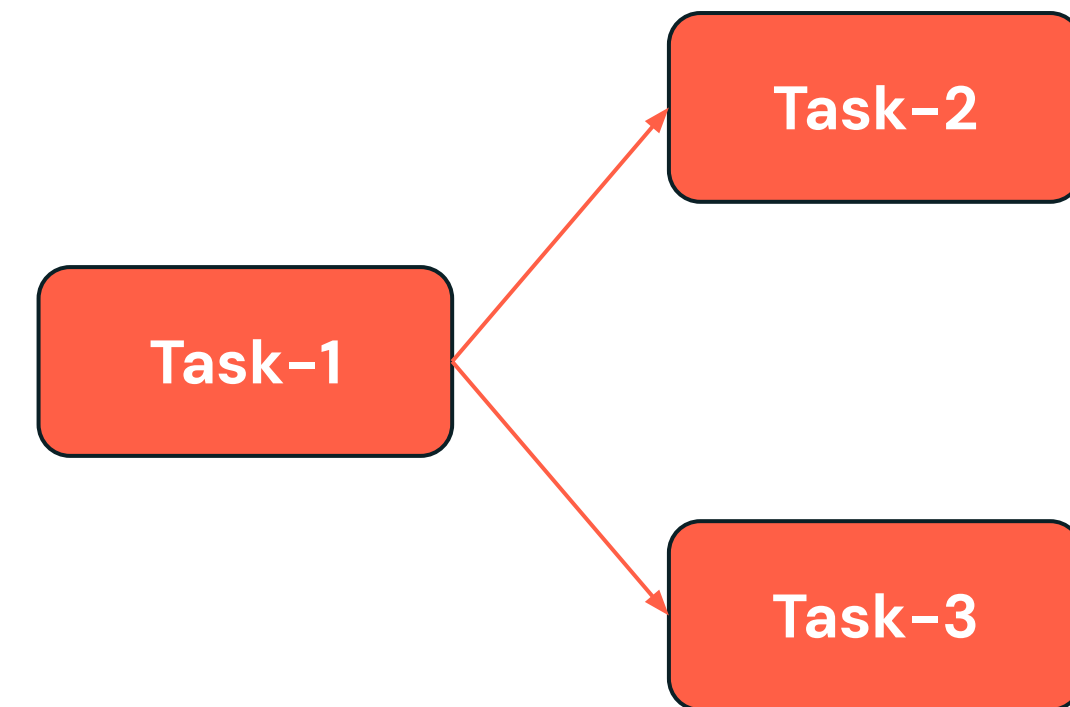


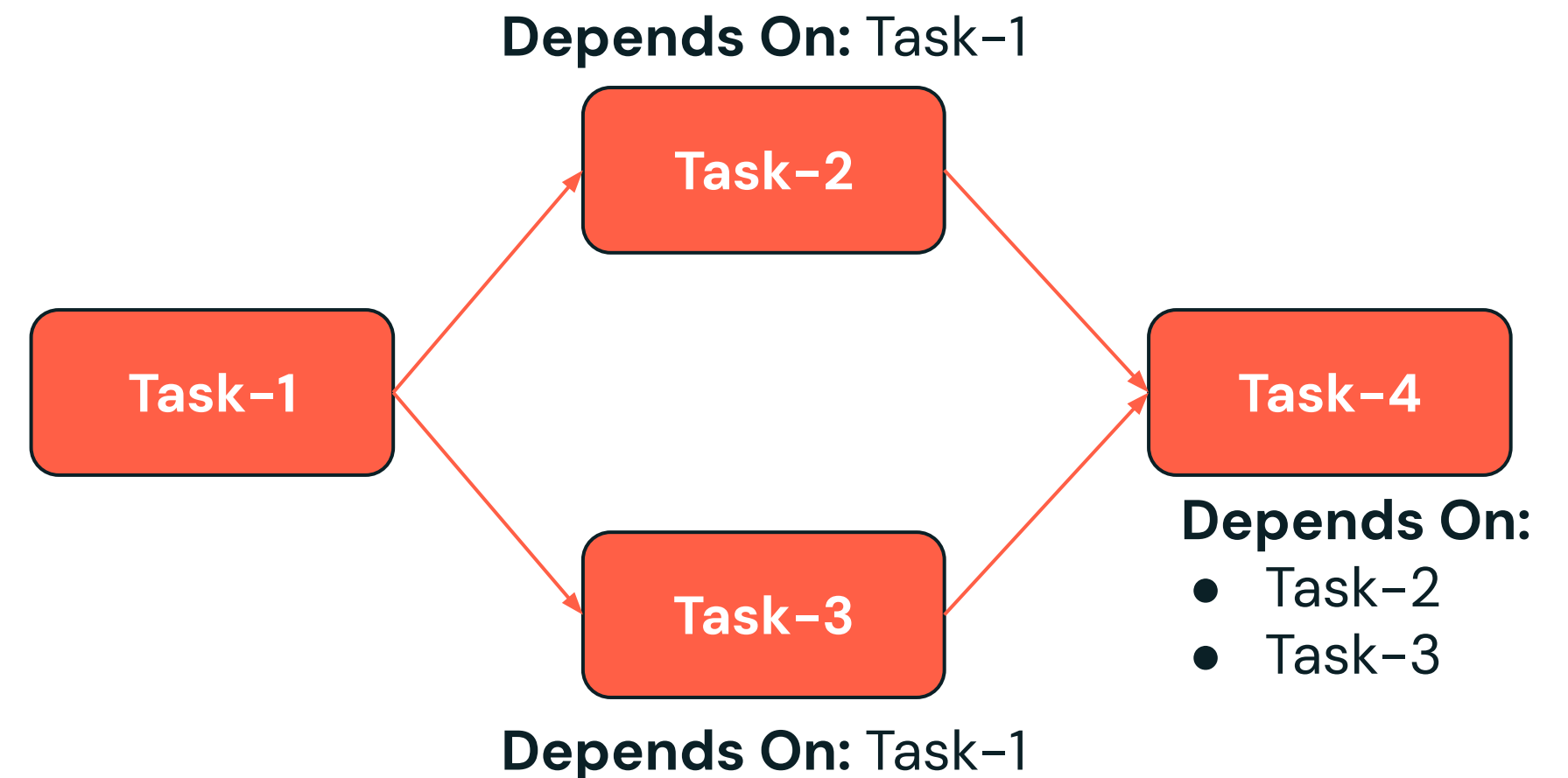
Diagram of a DAG  = vertices
edges = 



DAG and Task Orchestration

Task Orchestration

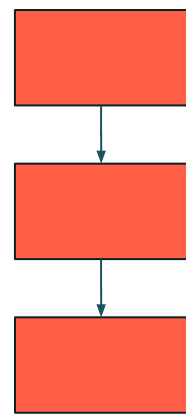
- Databricks Jobs supports **task orchestration**
 - The ability to run multiple tasks as a Directed Acyclic Graph (DAG)
 - Orchestrate tasks using the **Databricks UI** , **API**, **SDK** or **Databricks Asset Bundles**
- You can define the **order of execution of tasks** in a job by configuring task dependencies, creating a **DAG of task execution**



DAAG and Task Orchestration

Common Workloads Patterns

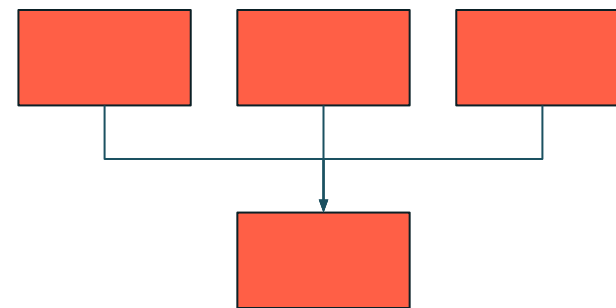
Sequence



Sequence

- Data transformation/processing/cleaning
- Bronze/silver/gold tables

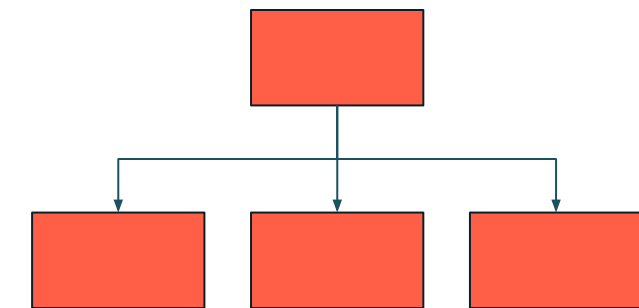
Funnel



Funnel

- Multiple data sources
- Data collection

Fan-out



Fan-out, star pattern

- Single data source
- Data ingestion and distribution

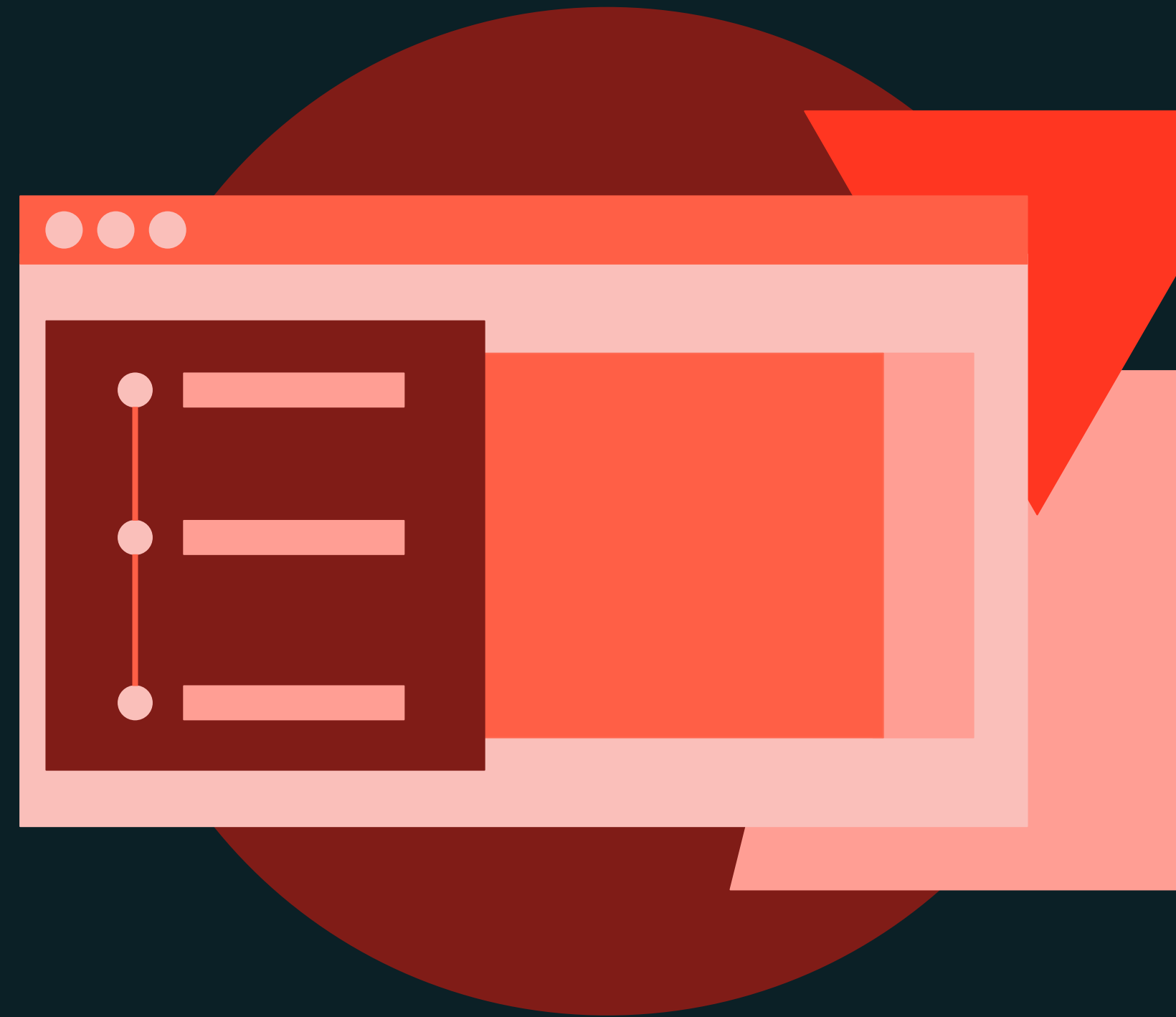




Lakeflow Jobs Core Components

DEMONSTRATION

Creating a Job Using Lakeflow Jobs UI



Notebook: 1 – Creating a Job Using Lakeflow Jobs UI



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



LakafLOW Jobs Core Components

LAB EXERCISE

Create a Job with Multiple Tasks



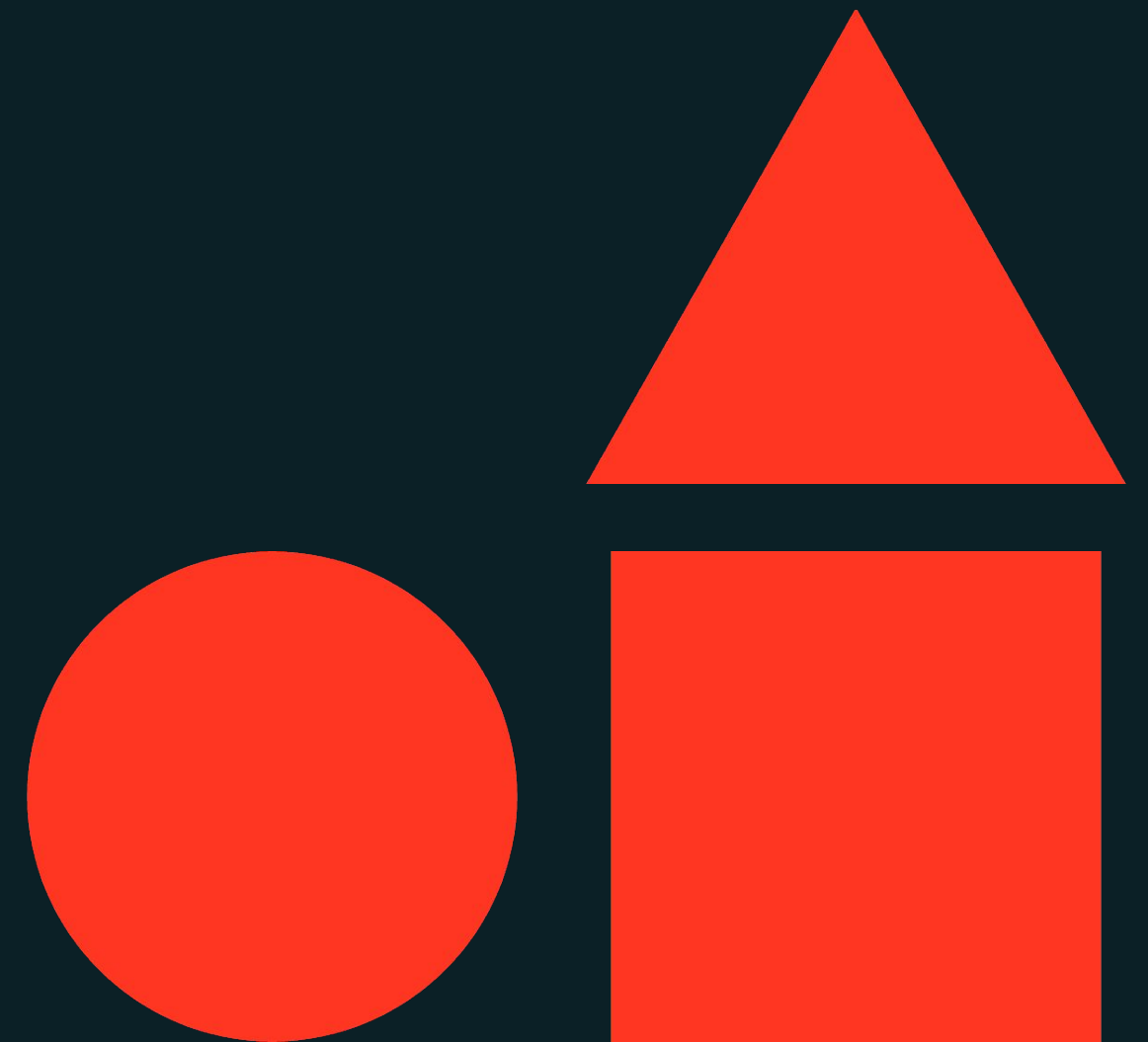
Notebook: 2 Lab – Create a Job with Multiple Tasks



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Creating and Scheduling Jobs

Deploy Workloads with Lakeflow Jobs



Section Learning Objectives

- Configure task parameters, notifications, and retry policies for Lakeflow Jobs.
- Set up and manage different job scheduling options, including scheduled, file arrival, and continuous triggers.
- Automate job execution using triggers to support time-based and event-driven workflows.
- Explore and apply scheduling options in the Databricks Lakeflow Jobs UI through hands-on demonstration.



Agenda

Creating and Scheduling Jobs

- **Common Task Configuration Options**
- **Jobs Scheduling Options**
- **Demo – Exploring Scheduling Options**





Creating and Scheduling Jobs

LECTURE

Common Task Configuration Options



Common Task Configuration Options

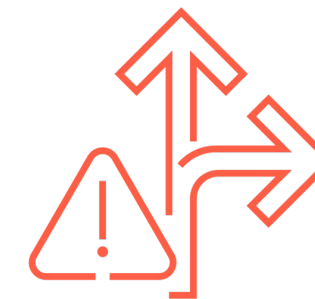
Overview



Parameters



Notification
Alerts



Retries



Common Task Configuration Options

Parameter Overview

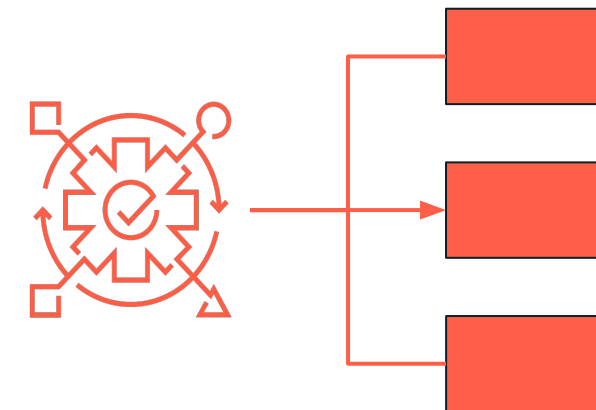
Task Parameters

A key-value pair or JSON array defined at the **task level**

Task

Job Parameters

A key-value pair defined at the **job level** and pushed down to tasks



Common Task Configuration Options

Parameter Overview

- **Task Parameters** are **key-value** pairs that lets you pass value into tasks
- Support **advanced orchestration** such as
 - Conditional execution
 - Looping
 - Passing context between tasks

Parameters ?

UI | JSON

environment	dev	×
threshold_value	1000	×

+ Add

Workflows > Jobs >

Lakeflow Jobs ☆

Runs Tasks

▶ □ 🗑

Unnamed task
Unspecified path

Source* ⓘ Workspace

Path* ⓘ Select Notebook

Compute* ⓘ Serverless

Dependent libraries ⓘ + Add

Parameters ⓘ

Key	Value
+ Add	

Notifications ⓘ + Add

Retries ⓘ Immediately, at most 3x (4 total attempts) ✎



Common Task Configuration Options

Parameter Overview

Setting parameter values

Option 1 - Using the UI



The screenshot shows the 'Parameters' section of a Databricks task configuration. It has a title 'Parameters' with a help icon. Below the title are two tabs: 'UI' (selected) and 'JSON'. There are two parameter rows. The first row has a label 'environment' and a value 'dev', with a close button 'X' to its right. The second row has a label 'threshold_value' and a value '1000', also with a close button 'X' to its right. At the bottom left, there is a '+ Add' button.

Option 2 - Databricks Utilities taskValues subutility (Python)

```
dbutils.jobs.taskValues.set(key = "environment",  
                           value = "dev")
```

Getting task parameter values

Getting task parameter with the task

```
env = dbutils.widgets.get("environment")  
  
threshold = dbutils.widgets.get("threshold_value")
```

Getting task parameter values from specific tasks

```
dbutils.jobs.taskValues.get(  
    taskKey="task-name",  
    key="parameter-name"  
)
```



Common Task Configuration Options

Notification Alerts

Task notifications

If you need to add or update a system destination configuration, reach out to your workspace administrator.

Supported destinations: Email, Microsoft Teams, PagerDuty, Slack, Webhook

Destination	Start	Success	Failure	Duration warning	Streaming backlog
samarth @databricks.com	☐	☒	☒	☐	☐

☐ Mute notifications for skipped runs

☐ Mute notifications for canceled runs

☐ Mute notifications until the last retry

+ Add notification

Cancel Save

Source*

Workspace

Path*

Select Notebook

Compute*

Serverless

Dependent libraries

+ Add

Parameters

UI JSON

Key

+ Add

Value

Notifications

+ Add

Retries

Immediately, at most 3x (4 total attempts)



Common Task Configuration Options

Notification Alerts

- **Multiple** supporting destinations as Emails, Teams, Slacks etc
- **Each task in a Job** could be configured **differently** to send notification
- Jobs triggers **notifications** when the task begins, completes or fails
- These notification could also be triggered in the case of **late Jobs**:
 - For duration threshold warning/timeout alerts
 - Streaming backlog(Issue with getting streaming data)

Task notifications

If you need to add or update a system destination configuration, reach out to your workspace administrator.

Supported destinations: Email, Microsoft Teams, PagerDuty, Slack, Webhook

Destination	Start	Success	Failure	Duration warning	Streaming backlog	
samarth.goel@databricks.com	☐	☒	☒	☐	☐	

☐ Mute notifications for skipped runs

☐ Mute notifications for canceled runs

☐ Mute notifications until the last retry

+ Add notification

CancelSave



Common Task Configuration Options

Retry Policy

A policy that determines when and how many times failed runs are **retried**

Retry Policy ✕

Jobs that fail are retried a number of times based on the following policy. You can specify a maximum number of attempts for a run and a minimal interval between attempts.

Retry at most **10 times (11 total attempts)** and wait **15** mins between retries

☐ Retry on timeout

- 5 times (6 total attempts)
- 6 times (7 total attempts)
- 7 times (8 total attempts)
- 8 times (9 total attempts)
- 10 times (11 total attempts) ✓**
- 20 times (21 total attempts)
- 30 times (31 total attempts)
- Unlimited

Cancel Confirm

Source* ⓘ Workspace

Path* ⓘ Select Notebook

Compute* ⓘ Serverless

Dependent libraries ⓘ + Add

Parameters ⓘ

Key	Value
+ Add	

Notifications ⓘ + Add

Retries ⓘ Immediately, at most 3x (4 total attempts) ✎





Creating and Scheduling Jobs

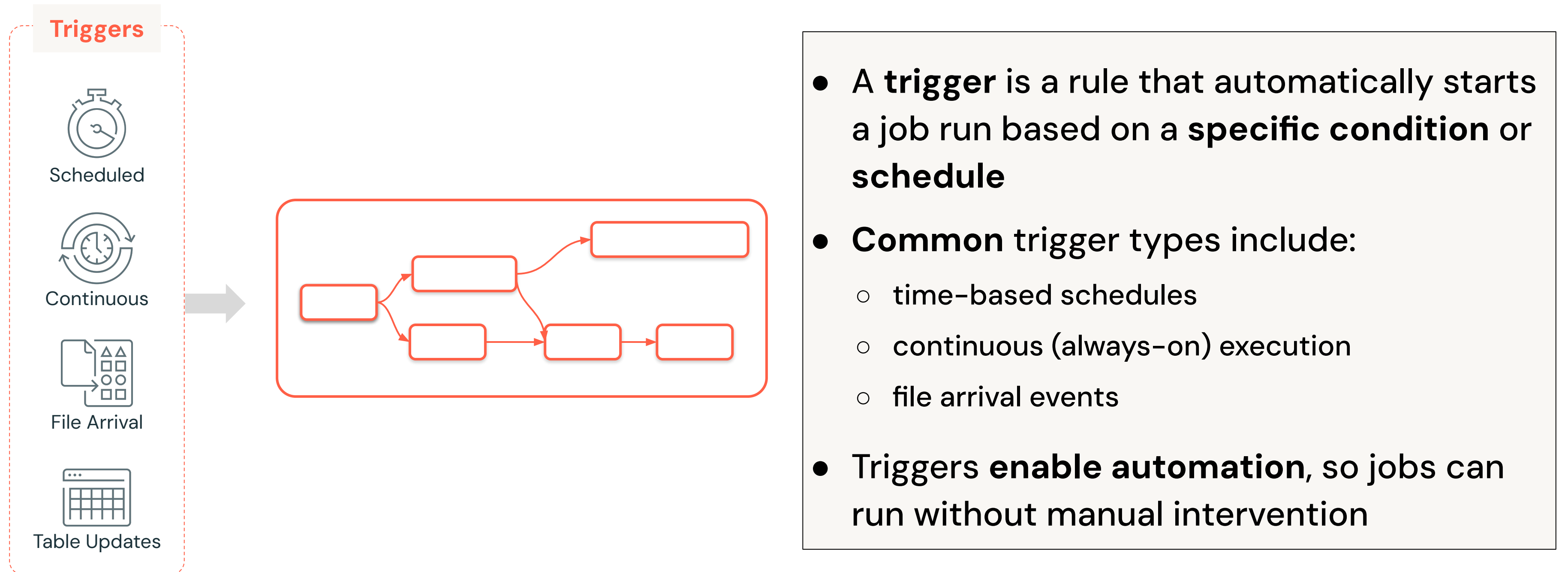
LECTURE

Job Schedules and Triggers



Job Schedules and Triggers

Overview



Job Schedules and Triggers

Types of Triggers

Scheduled Triggers

- **Automatically run jobs** at set times or intervals, such as hourly, daily using the UI
- You can schedule jobs using **cron expressions** for automated execution
- They help **automate** recurring jobs, ensuring jobs run consistently without manual intervention

Schedules & Triggers

Trigger Status

☒ Active

☐ Paused

Trigger type

Scheduled

Schedule type

Simple

Advanced

Schedule ⓘ

Every

Day

at

17

:

31

(UTC+00:00) UTC

☐ Show cron syntax

Cancel

Save



Job Schedules and Triggers

Types of Triggers

File Arrival Triggers

- **Automatically** trigger jobs **when new files are detected** in a specified storage location
- Enable **event-driven processing** to start data workflows upon **file arrival**
- Ideal for **automating jobs** with **unpredictable or irregular** data ingestion patterns

Schedules & Triggers

Trigger Status

☒ Active
☐ Paused

Trigger type

File arrival

File arrival triggers monitor cloud storage paths of up to 10,000 files for new files. These paths are either Unity Catalog volumes or external locations managed through Unity Catalog.

Storage location ⓘ

e.g. '/Volumes/mycatalog/myschema/myvolume/path_within_volume/' or 's3://bucket/path/'

Advanced ^

Minimum time between triggers ⓘ 00h 00m

Wait after last change ⓘ 00h 00m

Test trigger Cancel Save



Job Schedules and Triggers

Types of Triggers

Continuous Triggers

- **Continuously** runs jobs by starting a new run as soon as the previous one finishes or fails
- **Built-in retry** logic is automatically managed by Databricks

Schedules & Triggers

Trigger Status

☒ Active

☐ Paused

Trigger type

Continuous ⓘ

Cancel Save

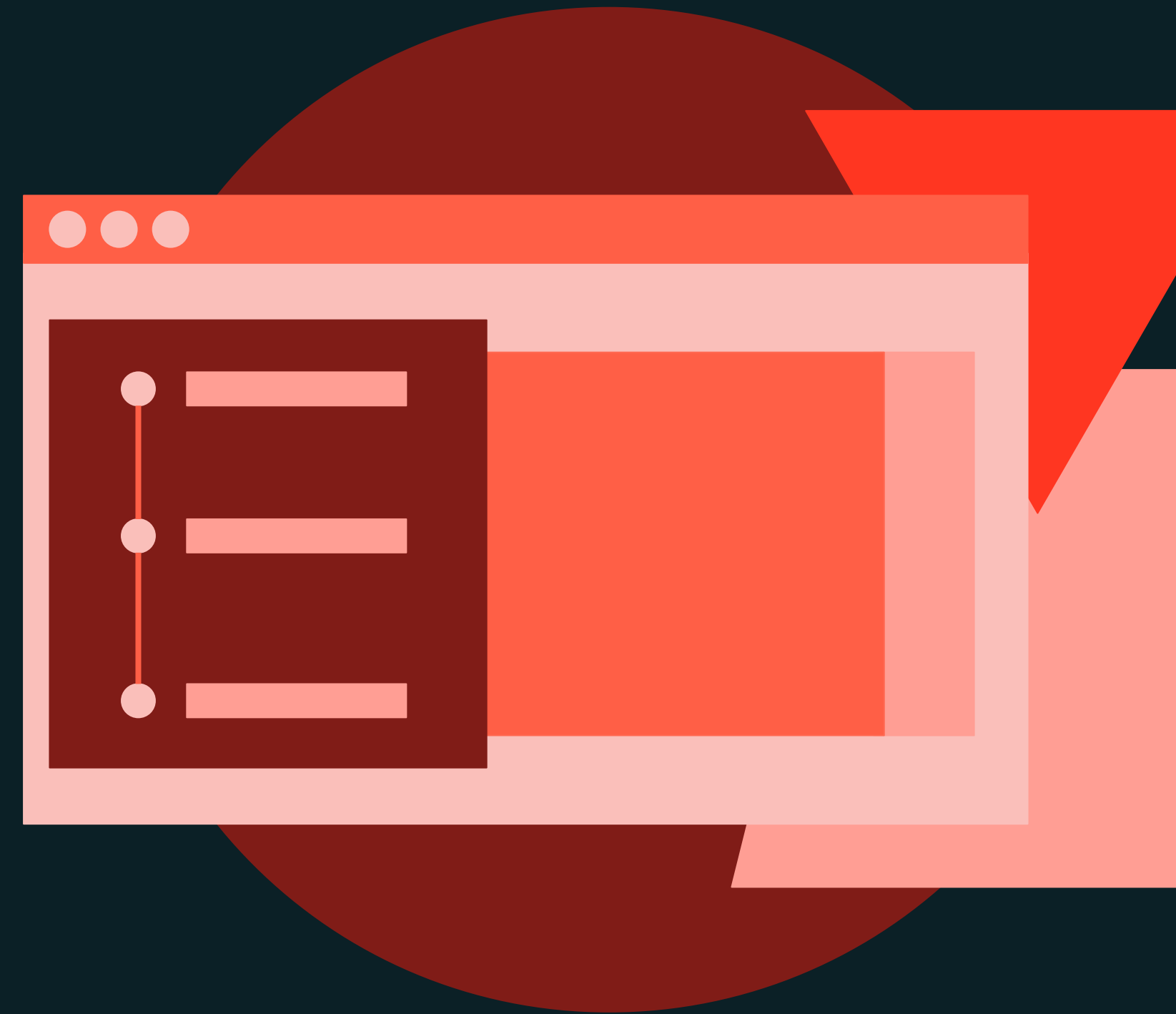




Creating and Scheduling Jobs

DEMONSTRATION

Explore Scheduling Options



Notebook: 3 – Explore Scheduling Options

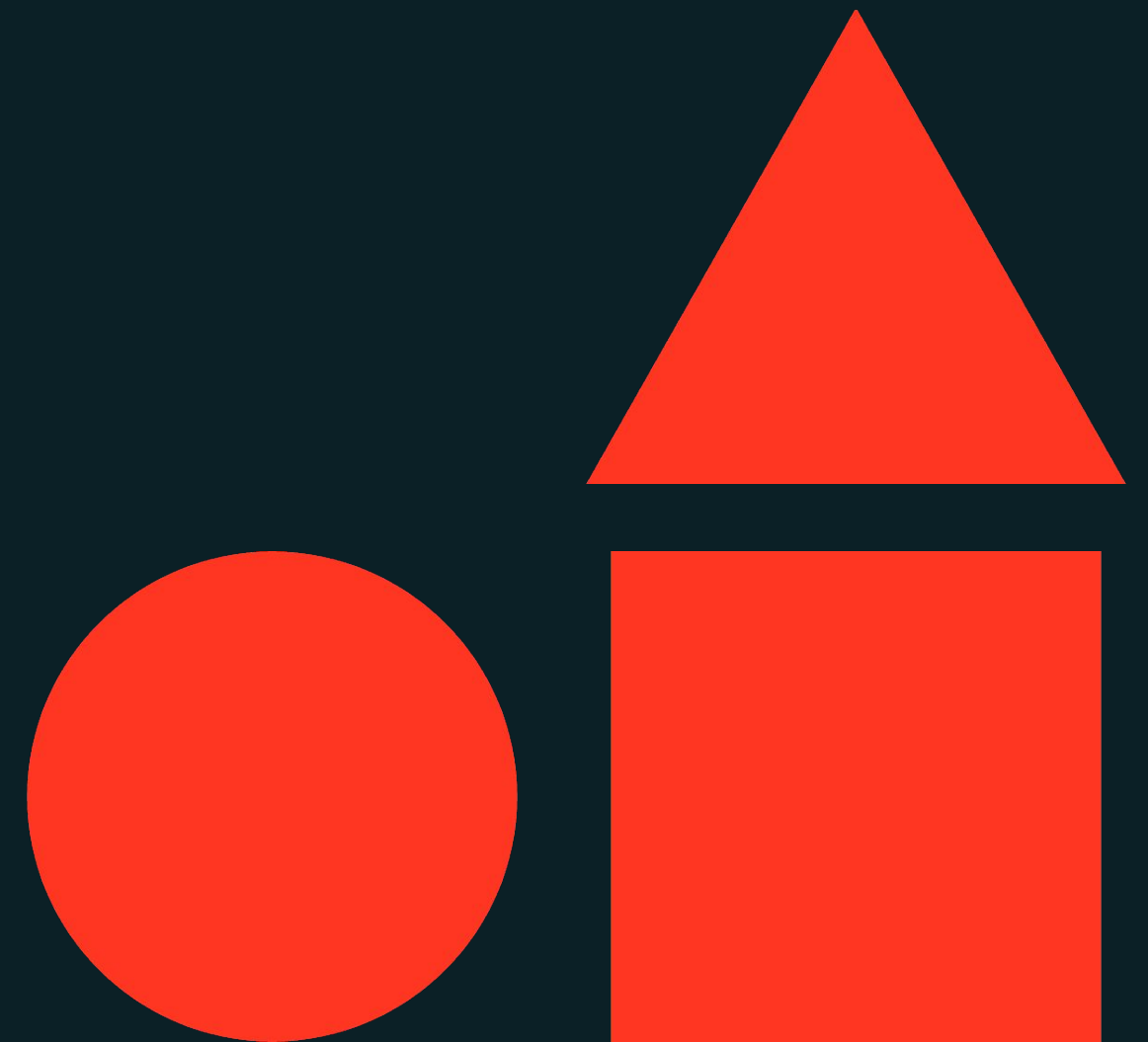


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Advance Lakeflow Jobs Features

Deploy Workloads with Lakeflow Jobs



Section Learning Objectives

- Implement conditional task execution using run-if dependencies and If/Else logic.
- Utilize the repair run feature to efficiently recover from failed job runs.
- Apply modular orchestration patterns to break down complex workloads into manageable components.
- Enhance workflow reliability and maintainability using advanced task controls.



Agenda

Advance Lakeflow Jobs Features

- **Control the Flow of Tasks**
- **Lakeflow Jobs in Production**
- **Best Practices**





Advance Lakeflow Jobs features

LECTURE

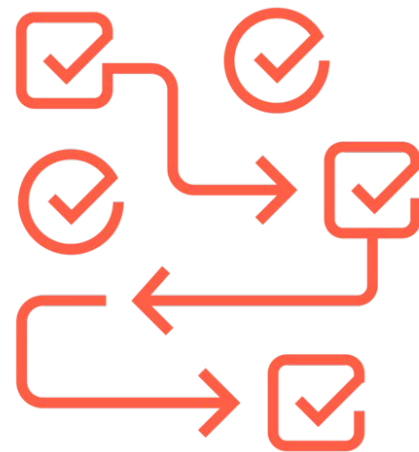
Control the Flow of Tasks



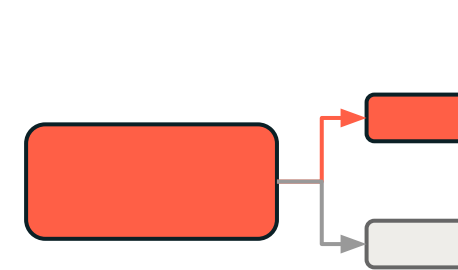
Control the Flow of Tasks

Overview

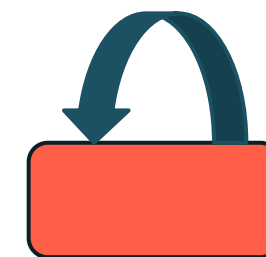
Run if Conditional Task Dependencies



Additional Task Flows



If/else



For each

Repair and Rerun

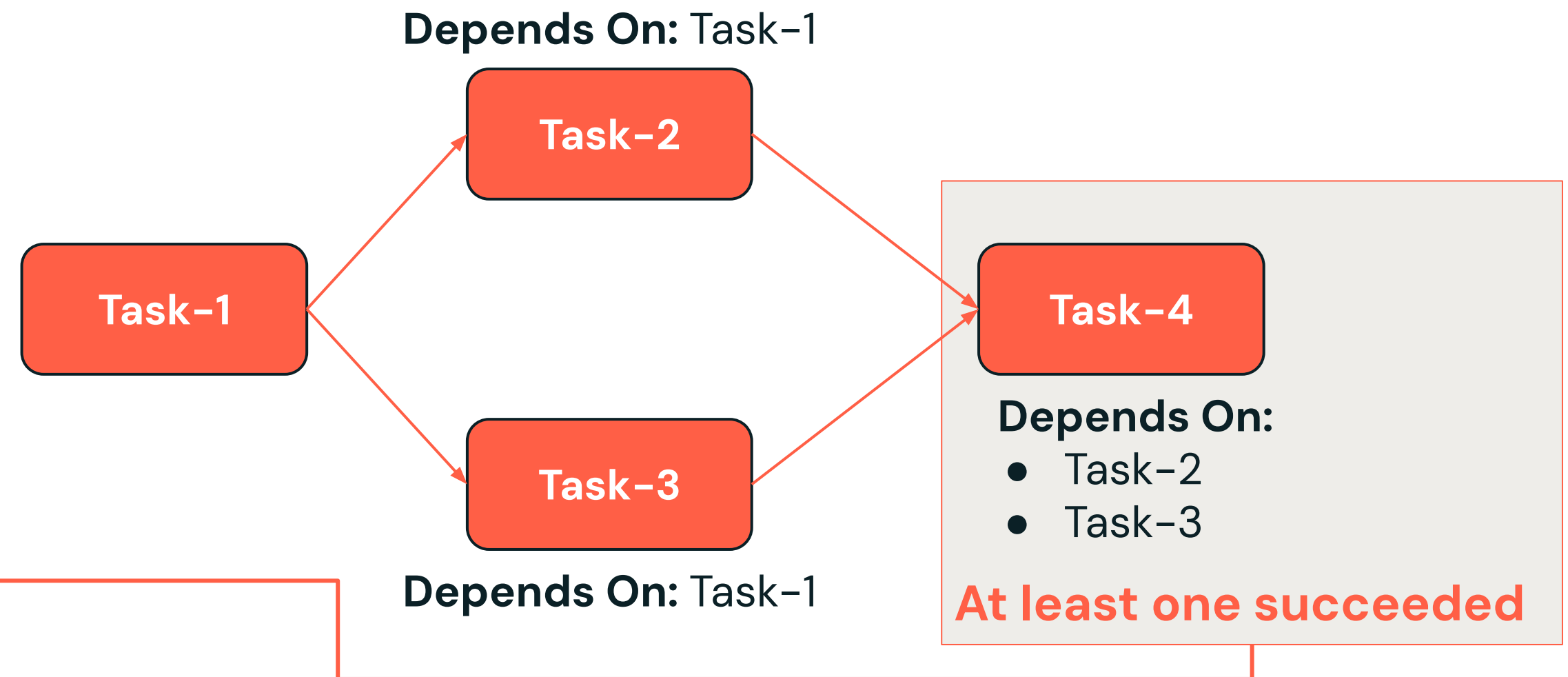


Control the Flow of Tasks

Overview

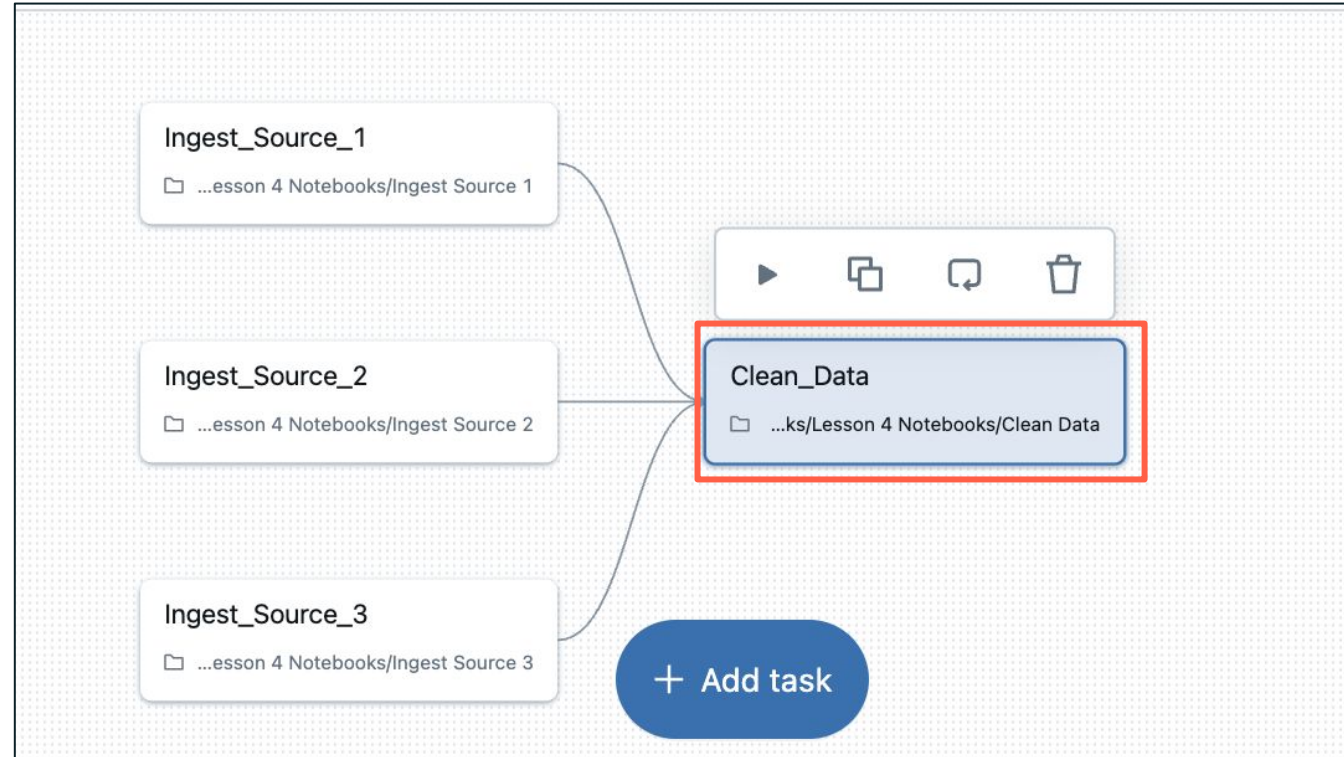
Run if Conditional Task Dependencies

- Control task execution based on outcomes of **upstream tasks**
- Supports a variety of **dependency conditions** like:
 - All succeeded
 - At least one succeeded
 - None failed
 - And more



Control the Flow of Tasks

Run if Conditional Task Dependencies



- Task dependencies provide **control flow logic** based on upstream task status
- Visually represented in the job DAG

Task name*	Clean_Data
Type*	Notebook
Source*	Workspace
Path*	...loads with Lakeflow Jobs/Task Notebooks/Lesson 4 Notebooks/Clean Data
Compute*	Serverless Autoscaling
Depends on	Ingest_Source_1 X Ingest_Source_2 X Ingest_Source_3 X
Run if dependencies	All succeeded

- ✓ All succeeded
All dependencies have executed and succeeded
- At least one succeeded
At least one dependency has succeeded
- None failed
None of the dependencies failed and at least one was executed
- All done
All dependencies have been completed
- At least one failed
At least one dependency failed
- All failed
All dependencies have failed



Control the Flow of Tasks

Run if Conditional Task Dependencies

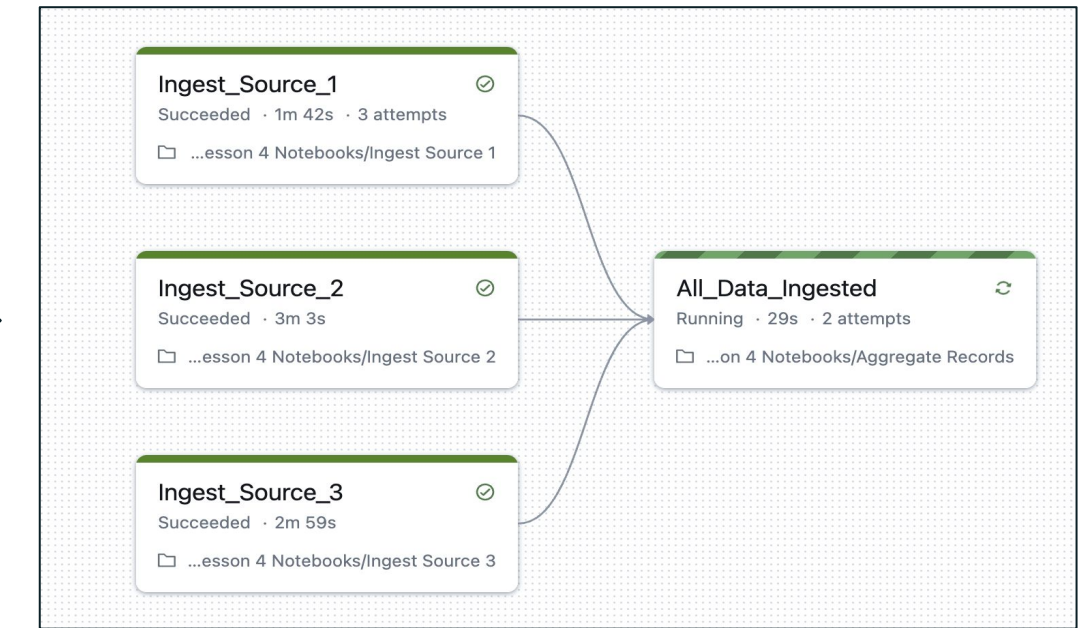
1. Job Initialization



2. Waiting for all task to complete



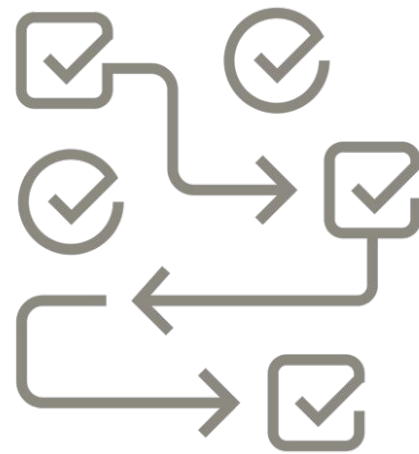
3. Dependent Task Started



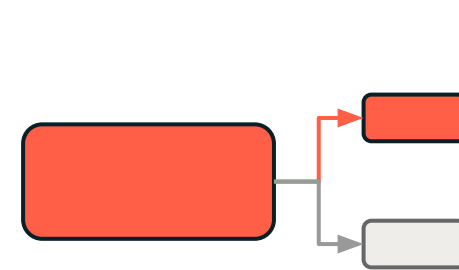
Control the Flow of Tasks

Overview

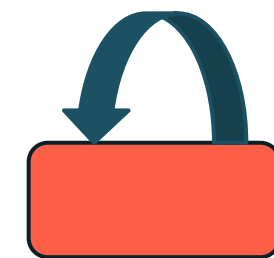
Run if Conditional Task Dependencies



Additional Task Flows

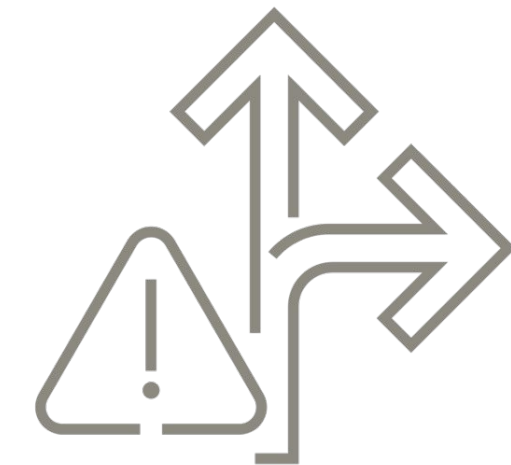


If/else



For each

Repair and Rerun



Control the Flow of Tasks

If/else Conditional Tasks

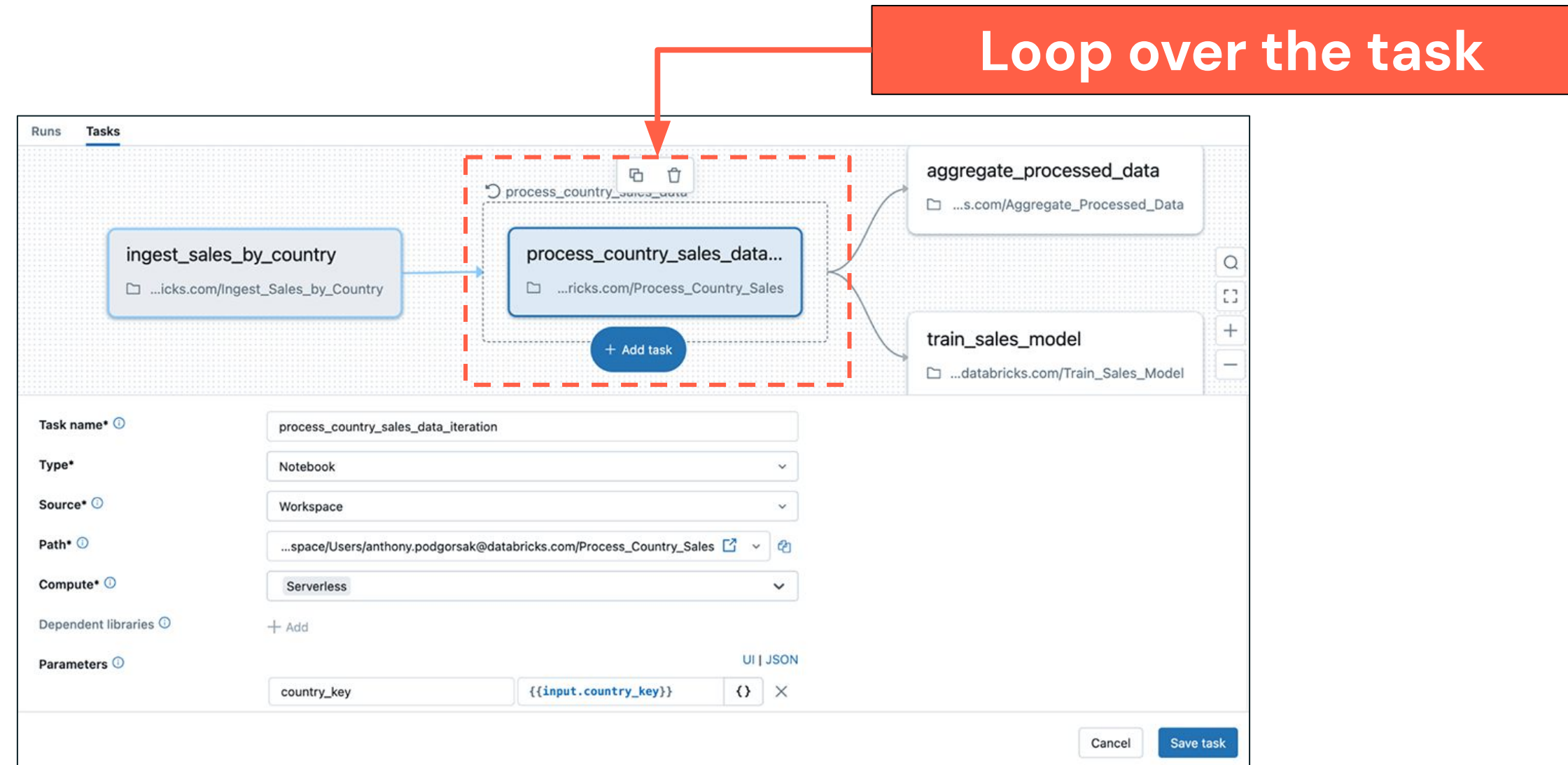


If/else Conditional Tasks

- Adds boolean conditional logic to your workflow based on task results
- Allows branching based on specific conditions (e.g., data quality checks, record counts)
- Uses boolean operators (`==`, `!=`, `>`, `>=`, `<`, `<=`) to evaluate expressions

Control the Flow of Tasks

For Each Tasks



For each tasks

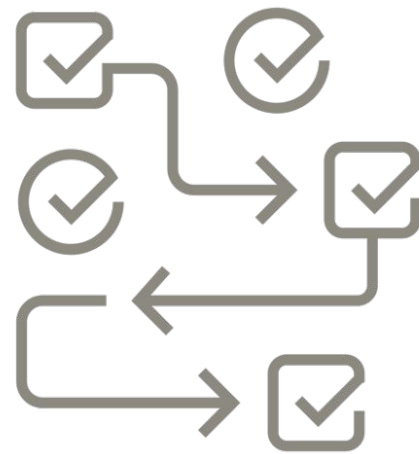
- Run another task in a loop, passing a different set of parameters to each iteration of the task
- Adding the **For each task** to a job requires defining two tasks: The For each task and a nested task.



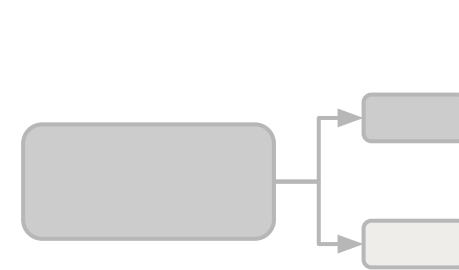
Control the Flow of Tasks

Overview

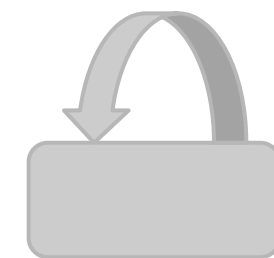
Run if Conditional Task Dependencies



Additional Task Flows



If/else



For each

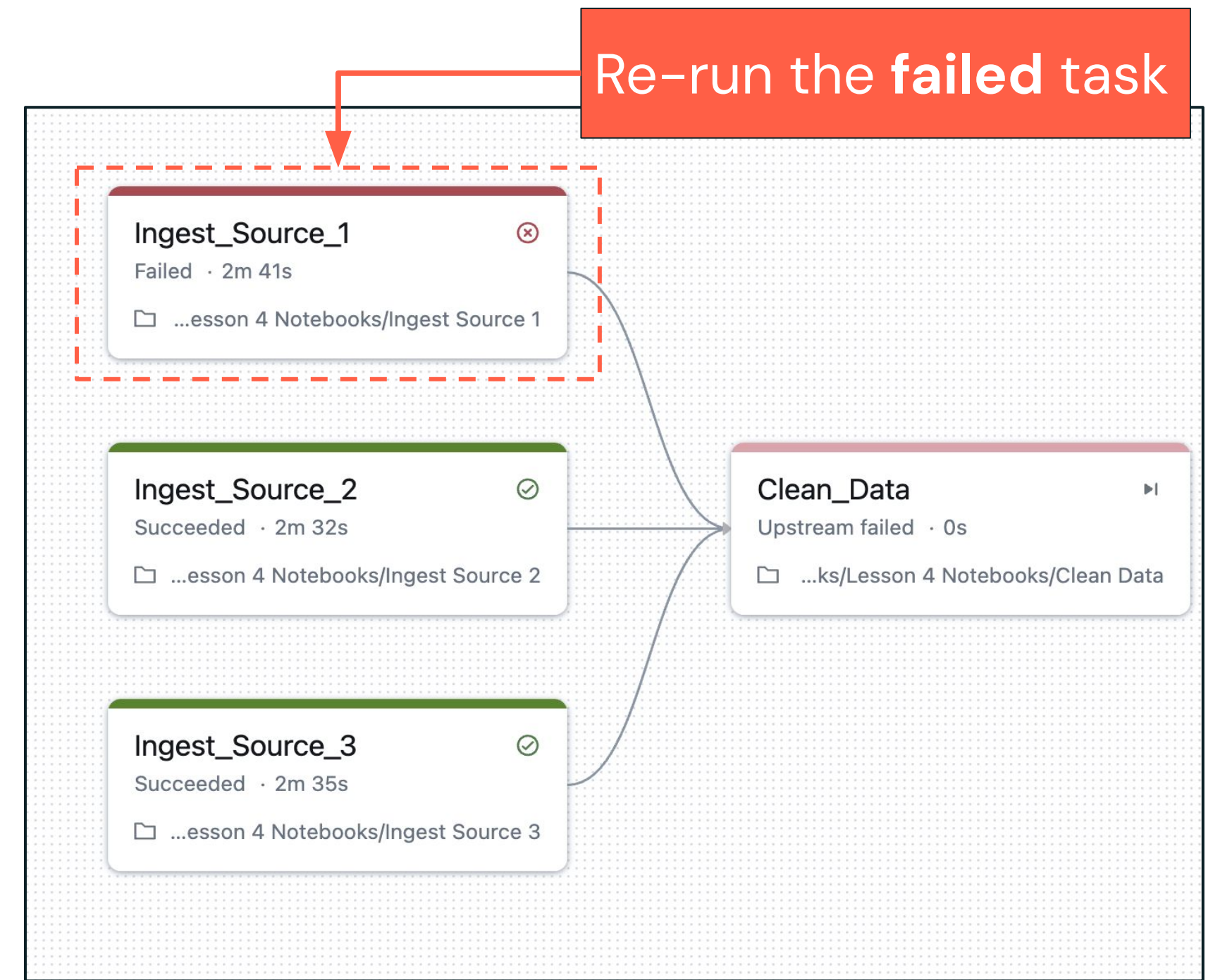
Repair and Rerun



Control the Flow of Tasks

Repair and Rerun

- **Repair** feature allows to **re-run only the failed task** and downstream tasks,
- **Reduces the time** and resources required to recover from unsuccessful job runs.



Control the Flow of Tasks

Repair and Rerun

Ingest_Source_1

Failed · 2m 41s

...esson 4 Notebooks/Ingest Source 1

Ingest_Source_2

Succeeded · 2m 32s

...esson 4 Notebooks/Ingest Source 2

Ingest_Source_3

Succeeded · 2m 35s

...esson 4 Notebooks/Ingest Source 3

Clean_Data

Upstream failed · 0s

...ks/Lesson 4 Notebooks/Clean Data

Send feedback

Delete job run

Repair run

Job run details

Job ID
654895606603408

Job run ID
8092008188237

Launched
Manually

Started
May 04, 2025, 03:38 PM

Ended
May 04, 2025, 03:41 PM

Duration
2m 42s

Queue duration
-

Status
Failed

Repair job run

Rerun tasks

The following tasks will be rerun:

- download_data_api
- combine_all
- notification

Parameters

Provide parameters for the repaired run. Please note that if you omit keys here, the default task parameters will be used.

UI

JSON

Notebook

Key	Value

Python

SQL Task

Key	Value

Cancel

Repair run

Allowing you to run only failed tasks, saving you time and money by only re-running the necessary tasks instead of the entire job

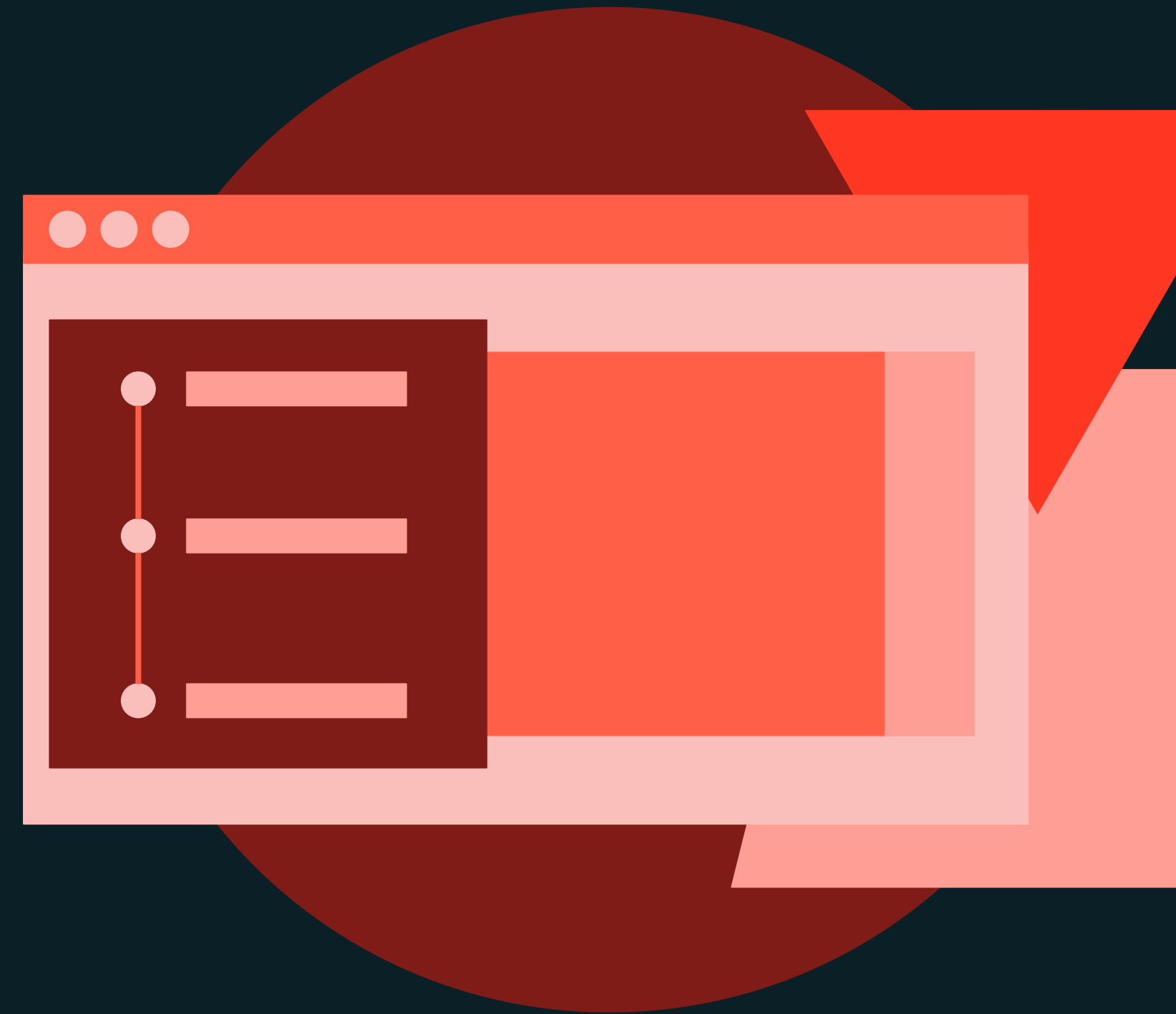




Advance Lakeflow Jobs Features

DEMONSTRATION

Controlling the Flow of a Job



Notebook: 4 – Controlling the Flow of a Job



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Advance Lakeflow Jobs Features

LECTURE

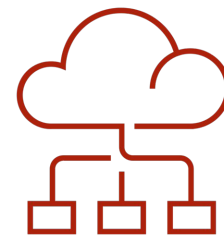
Lakeflow Jobs in Production



Lakeflow Jobs in Production

Selecting Compute

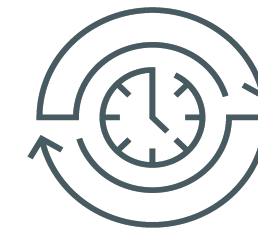
Interactive Clusters



It is best for performing ad-hoc analysis, data exploration, or development, but

- Costly for Job runs
- Limited Scalability
- Availability could be an issue because of parallel usage

Job Clusters



It is cheaper as they terminate when the job ends, reducing resource usage and costs. But,

- Start-up Latency
- Subject to cloud provider start-up time
- Maintenance Burden



Lakeflow Jobs in Production

Selecting Compute

Serverless Compute



Simplicity: You no longer need to choose the VM type. Customers do not need dedicated skilled DevOps folks

Lakeflow Jobs in Production

Selecting Compute

Serverless Compute



Simplicity: You no longer need to choose the VM type. Customers do not need dedicated skilled DevOps folks



High efficiency: Autoscaling and photon turned on default

Lakeflow Jobs in Production

Selecting Compute

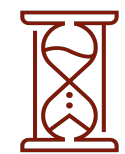
Serverless Compute



Simplicity: You no longer need to choose the VM type. Customers do not need dedicated skilled DevOps folks



High efficiency: Photon turned on default



Faster startup: VMs run in Databricks account. We ensure enough nodes are available based on historical data and ML

Lakeflow Jobs in Production

Selecting Compute

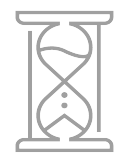
Serverless Compute



Simplicity: You no longer need to choose the VM type. Customers do not need dedicated skilled DevOps folks



High efficiency: Photon turned on default



Faster startup: VMs run in Databricks account. We ensure enough nodes are available based on historical data and ML



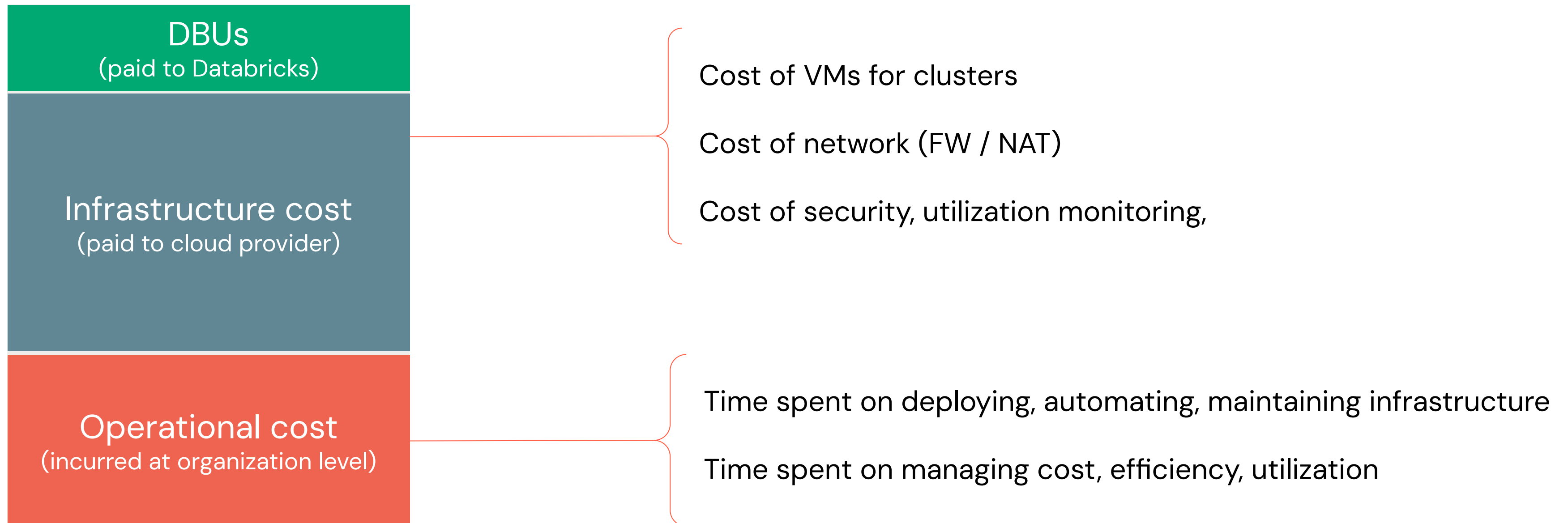
Reliability: Shielded from cloud disruptions



Lakeflow Jobs in Production

Pricing Structure

Classic



Lakeflow Jobs in Production

Pricing Structure

Classic

Serverless

DBUs

Infrastructure cost

Operational cost

TCO savings

DBUs
(Single bill that includes
infrastructure and operational
cost)

Value

Fully managed service – operationally simpler, more reliable

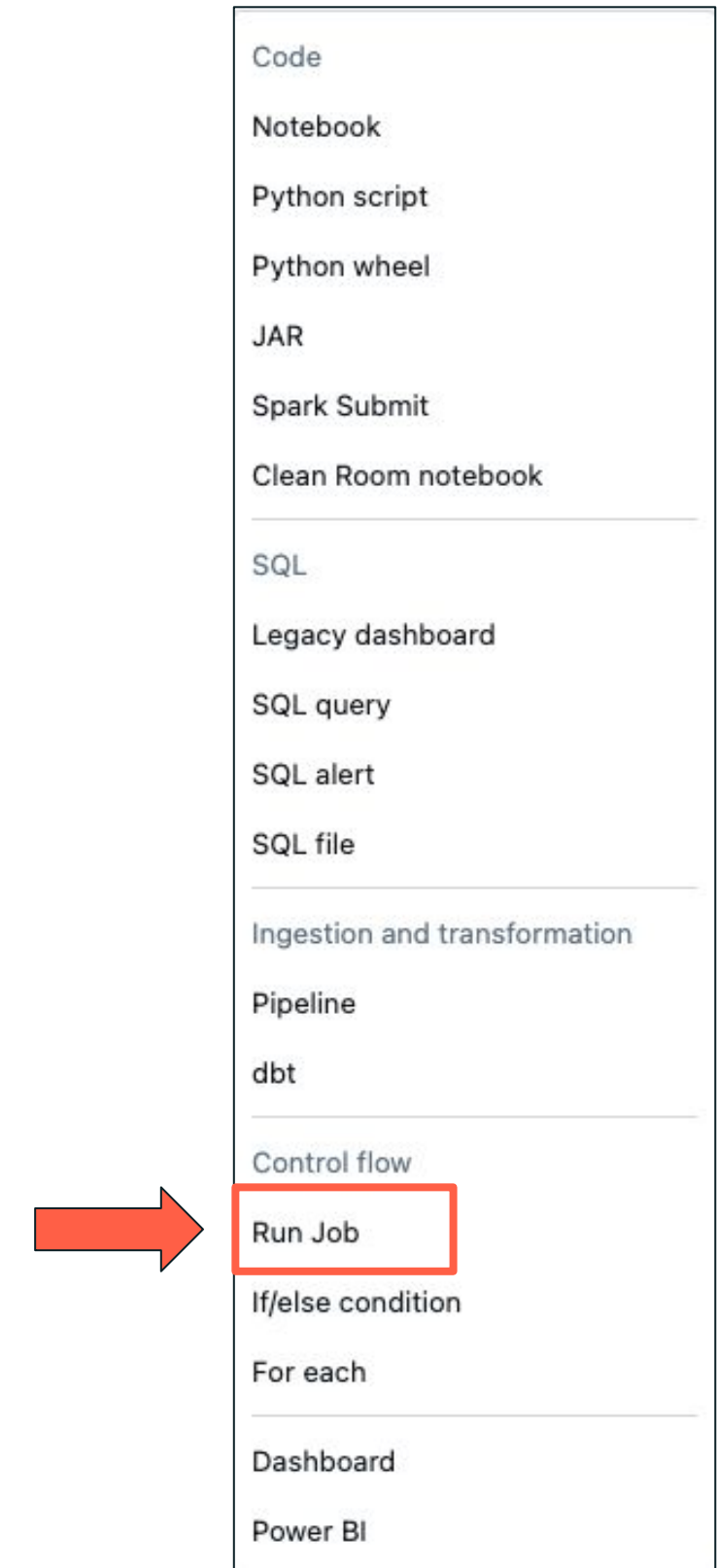
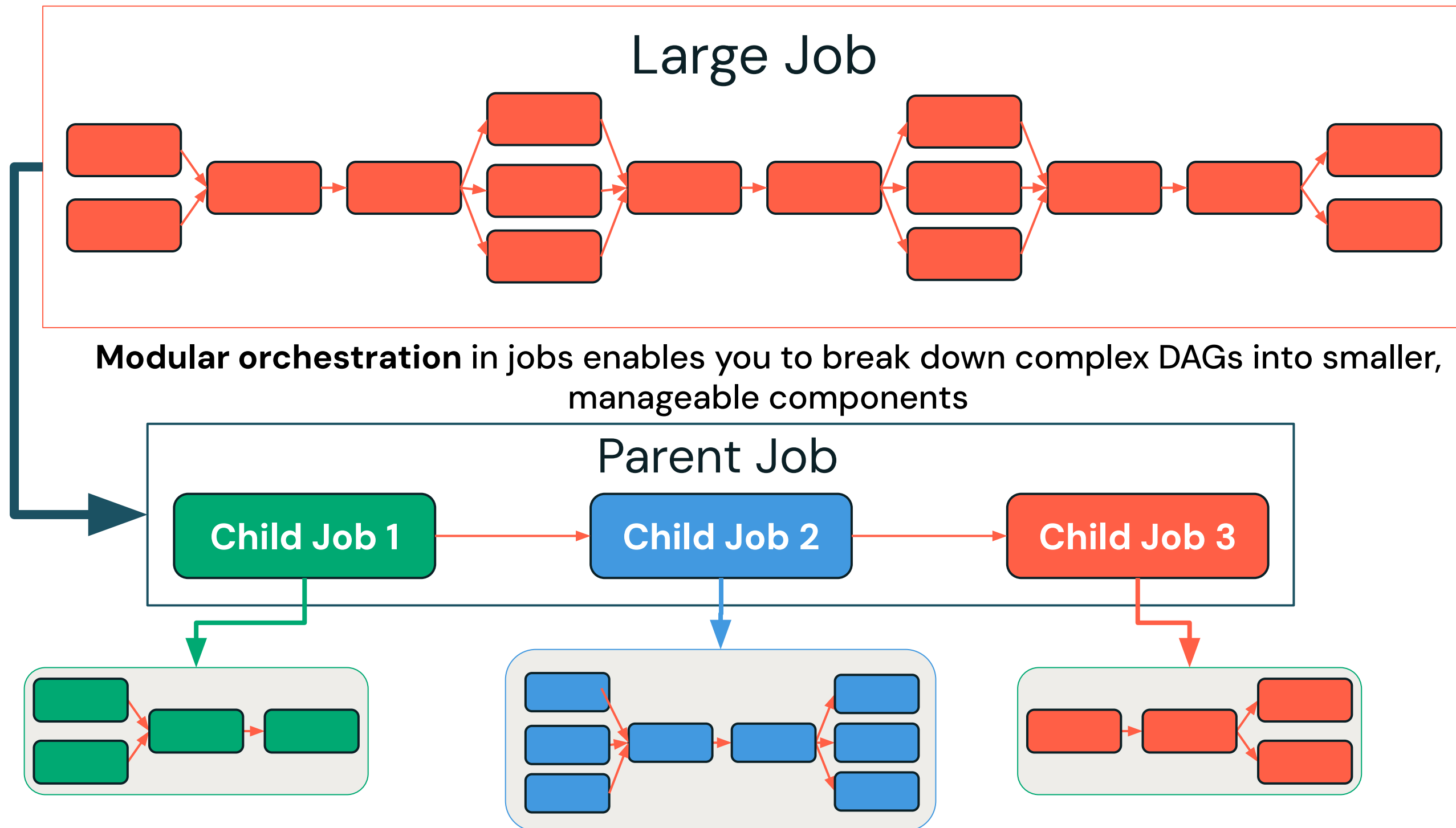
Fast clusters, auto-scaling – better user experience, lower cost

Out of the box performance and optimizations – lower overall TCO



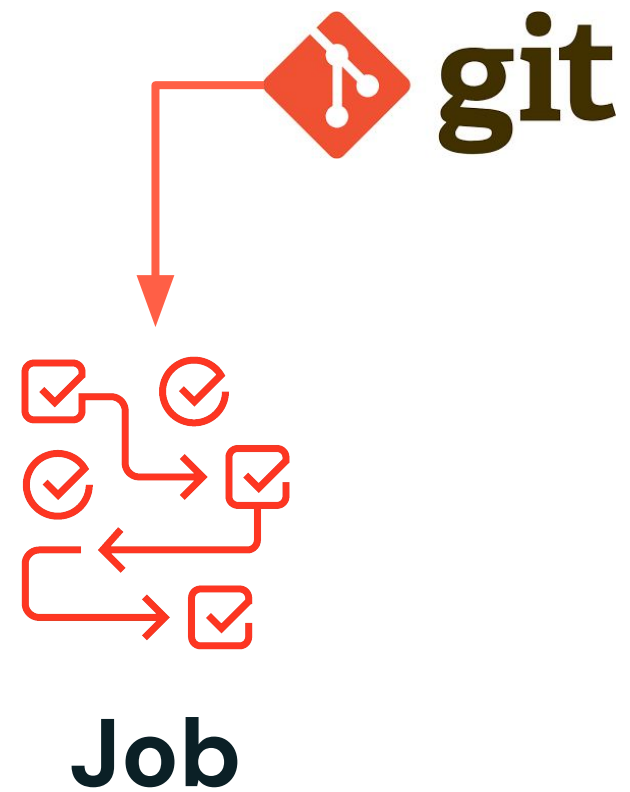
Lakeflow Jobs in Production

Modular Design in Databricks LakeFlow Jobs



Lakeflow Jobs in Production

Jobs and Git



Jobs supports execution of notebooks from Git

- This **prevents unintentional changes** to your **production** job, for example, when another user makes local edits in a 'Prod' repo or switches branches
- **Simplifying the job definition** process by having a single source of truth
- Makes **deployment** easy for notebooks through CI/CD

Lakeflow Jobs in Production

Jobs and Git Configuration Steps

Create task with Git provider as Source

1

A screenshot of the Lakeflow task configuration form. The 'Task name' field is 'Notebook_Git_Task'. The 'Type' dropdown is set to 'Notebook'. The 'Source' dropdown is set to 'Git provider', with an 'Edit' button and a dropdown arrow. Below the 'Source' dropdown is a section for 'Add a git reference' with a 'Workspace' option. The 'Cluster' dropdown is set to 'Git provider' with a checkmark. At the bottom, there is a 'Dependent libraries' section with a '+ Add' button.

Configure path to main notebook under repository root

2

A screenshot of the Lakeflow task configuration form, showing the 'Path' field. The 'Task name' is 'Notebook_Git_Task', 'Type' is 'Notebook', and 'Source' is 'Git provider (main)'. The 'Path' field is set to 'git_based_notebook_job/notebook_task'. The 'Cluster' dropdown is set to 'Shared Autoscaling EMEA 366 GB · 48 Cores · DBR 12.1 ML · Spark 3.3.1 · Sc...'. There is an 'Edit' button and a dropdown arrow next to the 'Source' field, and a link icon next to the 'Cluster' field.





Advance Lakeflow Jobs features

LAB EXERCISE

Modular Orchestration



Notebook: 5 Lab – Modular Orchestration



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Advance Lakeflow Jobs features

LECTURE

Best Practices



Best Practices

Automated/Job Clusters

- Always, *always* use Automated/Job Clusters for the production workloads
- Interactive/All Purpose Clusters are for development only

Databricks Runtime

- Always prefer to use the latest LTS Databricks Runtime (DBR)

Prefer Multi-Task Jobs (limited to 1000 Tasks in a Job)

- Depending on the pipeline requirements, each Task could run on its own cluster
- Dedicated, tailored Job clusters enable each task to run independently without resource sharing bottlenecks

Cluster Reuse

- Enables running tasks in a Databricks Job on the same cluster for more efficient cluster utilization and decreased job latency



Best Practices

Enable Photon

- The high-performance Databricks-native vectorized query engine
- Good to know: multipliers apply in the case of Photon (both for Interactive and Automated clusters)

Repair and re-run

- Instead of re-running all the tasks of a failed Job, repair and re-run helps running only the failed task onwards significantly reducing the time & resources required to recover
- Saves time and money

Late Jobs – Metric Threshold

- Tasks can trigger emails to stakeholders to proactively monitor and take action against late or long running jobs

Serverless Workflows

- [A new] Fully managed service operationally simpler and more reliable
- Faster clusters & auto-scaling capabilities with a better user experience for a lower cost and lower TCO



Best Practices

Triggers

- File Arrival Triggers
- Delta Table Triggers [FY24 Q4]

Run as SP

- “Run as” represents the identity the Job uses when it executes
- Allows control of what data the job has access to and allows to separate the access a job has from the access a user who runs (or manages) that job
- Ensure all the Jobs are owned by a Service Principal, instead of individual Data Engineers/DevOps folks

Modular Orchestration with Run Job Task type

- Reuse Jobs
- Breaking down huge Jobs into multiple smaller / manageable Tasks
- Easier testing
- Work across the teams

Orchestrate Data Analytics

- DB SQL workloads [Dashboards, queries and alerts] and DBT can be easily orchestrated with Lakeflow.



Best Practices

Cost attribution & chargeback

- Use Cluster Tags for cost attribution and chargeback

Enable notifications and alerts

- On emails
- Also with Webhooks (integrations with Slack, Microsoft Teams, PagerDuty, etc)

Control flow with Conditional Execution

- Task Dependencies with Run If and If/Else conditions

Parameterisation

- Job Parameters – Passed into each Task with behaviour based on the type
- Dynamic Value references– Special set of templated variables that provide introspective metadata about job and task
- Task Values – Custom parameters that can be shared between Tasks in a Job



Summary and Next Steps



Course Learning Objectives Recap

- Understand the role of Lakeflow Jobs within the Databricks ecosystem as a unified orchestration platform for data, analytics, and AI workloads.
- Design and implement data workloads using Directed Acyclic Graphs (DAGs), demonstrating the relationship between jobs, tasks, and dependencies.
- Configure various job scheduling options including manual, scheduled, file arrival, and continuous triggers to automate workflow execution.
- Implement advanced workflow features such as conditional task execution, run-if dependencies, and repair runs to create robust, fault-tolerant data pipelines.
- Apply best practices for production workloads including selecting appropriate compute options, implementing modular orchestration, and utilizing proper error handling techniques.



Next Steps

Additional resources for continuing the learning journey.

Data Engineering with Databricks

- Continue your learning through [self-paced](#) or [instructor-led](#) offerings
- The courses offer hands-on instruction in:
 - Databricks Data Science & Engineering Workspace
 - Databricks SQL
 - Delta Live Tables
 - Databricks Repos
 - Databricks Task Orchestration
 - Unity Catalog

Data Engineer Associate Certification

- Validate your data and AI skills on Databricks by earning a Databricks credential
- [Exam information](#) and [exam guide](#)
- The exam covers:
 - Data Intelligence Platform
 - ELT With Spark SQL and Python
 - Incremental Data Processing
 - Production Pipelines
 - Data Governance



